

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN 2

Data Fitting và Phương Pháp OLS

Nhóm 10 - Lớp: 24CTT3

Giảng viên hướng dẫn:

Lê Nhựt Nam

Võ Nam Thực Đoàn

Sinh viên thực hiện:

Nguyễn Duy Vũ - MSSV: 24120495

Phạm Văn Hữu Tài - MSSV: 24120435

Hà Lê Thanh Thúy - MSSV: 24120461

Trần Minh Tiến - MSSV: 24120465

Nguyễn Hữu Tường Vân - MSSV: 24120489

Mục lục

1 Phần 1: Data Fitting và OLS	5
1.1 Bài toán Data Fitting	5
1.1.1 Lý thuyết	5
1.1.2 Cài đặt thuật toán	5
1.1.3 Ví dụ	5
1.2 Phương pháp Ordinary Least Squares	5
1.2.1 Lý thuyết	5
1.2.2 Cài đặt thuật toán	6
1.2.3 Ví dụ	7
1.3 Minh họa định lý Gauss–Markov	8
1.3.1 Lý thuyết	8
1.3.2 Cài đặt thuật toán	8
1.3.3 Ví dụ	8
1.4 Hồi quy Ridge và Lasso	8
1.4.1 Lý thuyết	8
1.4.2 Cài đặt thuật toán	9
1.4.3 Ví dụ	10
1.5 Phân tích phần dư	10
1.5.1 Lý thuyết	10
1.5.2 Cài đặt thuật toán	11
1.5.3 Ví dụ	11
1.6 Đánh giá chéo	11
1.6.1 Lý thuyết	11
1.6.2 Cài đặt thuật toán	12
1.6.3 Ví dụ	12
2 Phần 2: Ứng dụng Data Fitting vào dữ liệu thực tế	13
2.1 Giới thiệu	13
2.2 Phân tích khám phá dữ liệu	13
2.2.1 Mô tả dataset	13
2.2.2 Thống kê mô tả	14
2.2.3 Phân tích missing values	15

2.2.4	Phân tích phân phối	17
2.2.5	Phát hiện và xử lý outlier	19
2.2.6	Phân tích tương quan	21
2.3	Tiền xử lý dữ liệu	22
2.3.1	Chia tập huấn luyện và kiểm thử	22
2.3.2	Xử lý dữ liệu khuyết thiếu	23
2.3.3	Mã hóa biến phân loại	24
2.3.4	Chuẩn hóa dữ liệu	25
2.3.5	Phân tích đa cộng tuyến	25
2.4	Xây dựng mô hình	27
2.4.1	Các chỉ số đánh giá	27
2.4.2	Model 1: OLS Full - Hồi quy tuyến tính trên tất cả đặc trưng	28
2.4.3	Model 2: OLS Selected - Hồi quy trên đặc trưng đã chọn lọc	28
2.4.4	Model 3: Ridge Regression với Cross-Validation	29
2.4.5	Model 4: Lasso Regression với Cross-Validation	31
2.5	So sánh và đánh giá mô hình	34
2.5.1	Bảng tổng hợp kết quả trên tập kiểm thử	34
2.5.2	Phân tích kết quả	34
2.6	Phân tích phần dư	36
2.6.1	Residuals vs Fitted	36
2.6.2	Normal Quantile-Quantile Plot	37
2.6.3	Scale-Location Plot	38
2.6.4	Cook's Distance	39
2.7	Phân tích tầm quan trọng đặc trưng	40
2.8	Kết luận	42
2.8.1	Tóm tắt kết quả	42
2.8.2	So sánh Ridge và Lasso - Bài học về Regularization	42
2.8.3	Đặc trưng quan trọng nhất - Diễn giải thị trường	43
2.8.4	Hạn chế của mô hình - Phân tích nguyên nhân R^2 thấp	43
3	Kết luận	45
3.1	Kết quả đạt được	45
3.2	Bài học rút ra	45
3.3	Khó khăn gặp phải	46

4 Bảng phân công công việc	47
Tài liệu tham khảo	48

1 | Phần 1: Data Fitting và OLS

1.1 | Bài toán Data Fitting

1.1.1 | Lý thuyết

Mục tiêu của data fitting là tìm một hàm xấp xỉ tốt nhất mô tả quan hệ giữa biến đầu vào và biến đầu ra. Trong hồi quy tuyến tính, mô hình được viết dưới dạng

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i$$

hay tương đương ở dạng ma trận:

$$y = X\beta + \varepsilon$$

Ở đây, X là ma trận thiết kế; cột đầu tiên thường được thêm bằng 1 để biểu diễn hệ số chặn β_0 . Khi mô hình tuyến tính được giả thiết đúng, bài toán fitting trở thành nền tảng cho ước lượng tham số, kiểm định giả thuyết và đánh giá chất lượng dự đoán.

1.1.2 | Cài đặt thuật toán

Trong notebook, phần khởi tạo dữ liệu và tiền xử lý được tổ chức trong các hàm tiện ích như `add_intercept`, `_as_1d_array` và `_as_2d_array`. Các hàm này đảm bảo đầu vào luôn có dạng phù hợp trước khi đưa vào mô hình.

1.1.3 | Ví dụ

Với dữ liệu giả lập $y = 2x + 1$, mô hình có thể học chính xác quan hệ tuyến tính giữa đầu vào và đầu ra. Đây là ví dụ cơ bản để kiểm tra xem toàn bộ pipeline từ dữ liệu đến ước lượng có hoạt động đúng hay không.

1.2 | Phương pháp Ordinary Least Squares

1.2.1 | Lý thuyết

Ordinary Least Squares (OLS) là phương pháp kinh điển để ước lượng tham số trong hồi quy tuyến tính bằng cách tối thiểu hóa tổng bình phương phần dư:

$$RS(\beta) = \|y - X\beta\|_2^2 = \sum_{i=1}^n (y_i - x_i^T \beta)^2$$

Lấy đạo hàm theo β và cho bằng 0, ta thu được phương trình chuẩn:

$$X^T X \hat{\beta} = X^T y$$

Nếu $X^T X$ khả nghịch, nghiệm OLS duy nhất là

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y$$

Từ nghiệm này, giá trị dự đoán và phần dư được xác định bởi

$$\hat{y} = X\hat{\beta}, \quad \hat{e} = y - \hat{y}$$

Ma trận chiếu (hat matrix) được cho bởi

$$H = X(X^T X)^{-1} X^T$$

và thỏa mãn các tính chất quan trọng như đối xứng và idempotent. Phương sai nhiễu được ước lượng bởi

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1}$$

Ngoài ra, các chỉ số thường dùng trong đánh giá mô hình là

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

và

$$\overline{R}^2 = 1 - \frac{n-1}{n-p}(1 - R^2)$$

Suy luận thống kê cho từng hệ số dùng thống kê t:

$$t_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{((X^T X)^{-1})_{jj}}}$$

và kiểm định tổng thể dùng thống kê F.

1.2.2 | Cài đặt thuật toán

Các hàm chính trong `ols_implementation.py`:

1. `add_intercept(X)`: nhận X dạng 1D / 2D, trả về ma trận thiết kế có cột 1 (shape: $n \times (p + 1)$).
2. `_as_1d_array(y)` / `_as_2d_array(X)`: chuẩn hóa input, chuyển đổi và kiểm tra chiều dữ liệu.

3. `ols_fit(X, y, add_intercept_column=True)` :
 - Mục đích: tính nghiệm OLS theo công thức đóng.
 - Các bước chính: chuẩn hóa đầu vào, thêm cột chặn (nếu cần), tính $X^T X$ và $X^T y$, giải hệ tuyến tính cho $\hat{\beta}$ bằng `np.linalg.solve`, tính $\hat{y}$, residuals, rss và $\hat{\sigma}^2$ (với $df = n - p - 1$).
 - Trả về: dict chứa $\hat{\beta}$, $\hat{\sigma}^2$, $\hat{y}$, residuals, design_matrix, `xtx`, `xtx_inv`, `n_samples`, `n_params`, `df_resid`.
 - Lưu ý: ném `ValueError` nếu $X^T X$ suy biến hoặc $df \leq 0$.
4. `hat_matrix(X, add_intercept_column=True)` : xây ma trận chiều $H = X(X^T X)^{-1} X^T$, kiểm tra đối xứng và idempotent, trả về H và rank.
5. `model_metrics(y, y_hat, p)` : tính rss, tss, r^2 , adjusted r^2 và f_statistic (với tham số $p =$ số biến giải thích, không kể intercept).
6. `coef_inference(X, y, beta_hat, sigma2_hat, add_intercept_column=True, alpha=0.05)` :
 - Mục đích: ước lượng sai số chuẩn của hệ số, tính t-stat, p-value và khoảng tin cậy (dựa trên phân phối Student-t).
 - Trả về: `standard_errors`, `t_statistics`, `p_values`, `confidence_intervals`, `df_resid`.
7. `vif(X, add_intercept_column=True)` : tính Variance Inflation Factor cho từng biến (loại trừ intercept nếu có), dùng OLS phụ để ước lượng R^2 từng biến trên phần còn lại.
8. `verify_ols_with_sklearn(X, y, add_intercept_column=True)` : chạy `sklearn.linear_model.LinearRegression` trên cùng dữ liệu và so sánh $\hat{\beta}$ và $\hat{y}$, để kiểm chứng tính đúng đắn của cài đặt.

Ví dụ sử dụng (tóm tắt trong notebook): khởi tạo X một cột, $y = 2x + 1$, gọi `ols_fit(X, y)` và kiểm tra $\hat{\beta}$, residuals, sau đó gọi `coef_inference` trên dữ liệu có nhiễu để thu `standard_errors` và `p_values`.

1.2.3 | Ví dụ

Với bộ dữ liệu giả lập $y = 2x + 1$, kết quả thực nghiệm cho thấy hệ số ước lượng OLS đúng bằng $[1, 2]$, phần dư bằng 0, $R^2 = 1$ và ma trận hat thỏa mãn tính idempotent. Đây là ví dụ minh họa trường hợp mô hình khớp hoàn hảo.

1.3 | Minh họa định lý Gauss–Markov

1.3.1 | Lý thuyết

Định lý Gauss–Markov phát biểu rằng, dưới các giả thiết chuẩn của mô hình tuyến tính cổ điển, OLS là ước lượng tuyến tính không chệch tốt nhất (BLUE). Điều này có nghĩa là trong lớp các ước lượng tuyến tính không chệch, OLS có phương sai nhỏ nhất.

Giả thiết thường dùng gồm:

- Mô hình tuyến tính.
- Không có đa cộng tuyến hoàn hảo.
- Sai số có kỳ vọng bằng 0 khi đã biết X .
- Phương sai không đổi và sai số không tương quan.

1.3.2 | Cài đặt thuật toán

Trong notebook, phần minh họa Gauss–Markov sử dụng mô phỏng Monte Carlo: ta cố định ma trận thiết kế, sinh nhiều mẫu nhiễu Gaussian rồi tính trung bình nhiều lần ước lượng OLS. Đồng thời, notebook còn dựng một ước lượng tuyến tính không chệch khác để so sánh phương sai với OLS.

1.3.3 | Ví dụ

Kết quả mô phỏng cho thấy trung bình các ước lượng OLS rất gần với tham số thật, còn trace của phương sai thực nghiệm và phương sai lý thuyết của OLS đều nhỏ hơn ước lượng thay thế. Điều này trực tiếp minh họa kết luận của định lý Gauss–Markov.

1.4 | Hồi quy Ridge và Lasso

1.4.1 | Lý thuyết

Khi dữ liệu có nhiều đặc trưng hoặc xảy ra đa cộng tuyến, OLS có thể kém ổn định. Khi đó ta thêm regularization vào hàm mục tiêu.

Ridge Regression (L_2) được định nghĩa bởi

$$\hat{\beta}_{\text{ridge}} = \operatorname{argmin}_{\beta} (\|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2)$$

Nghiem đóng của Ridge là

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

Ridge làm co nhỏ hệ số nhưng thường không đưa hệ số về đúng 0.

Lasso Regression ($L1$) được định nghĩa bởi

$$\hat{\beta}_{\text{lasso}} = \operatorname{argmin}_{\beta} (\|y - X\beta\|_2^2 + \lambda \|\beta\|_1)$$

Lasso không có nghiệm đóng đơn giản; thuật toán thường dùng là coordinate descent hoặc các phương pháp tối ưu gần đúng. Do dùng chuẩn $L1$, Lasso có khả năng tạo nghiệm thưa, tức là đẩy một số hệ số về 0.

1.4.2 | Cài đặt thuật toán

Các hàm chính trong `ridge_lasso.py` :

1. `ridge_fit(X, y, lam, add_intercept_column=True)` :
 - Mục đích: nghiệm đóng Ridge theo công thức (không phạt intercept khi `add_intercept_column=True`).
 - Cách làm: thêm cột chặn nếu cần, xây penalty (ma trận đơn vị với hàng cột đầu = 0 khi không phạt intercept), giải $(X^T X + \lambda \text{penalty})\beta = X^T y$ bằng `np.linalg.solve`.
 - Trả về: $\hat{\beta}$, $\hat{y}$, λ , residuals, rss, design_matrix.
2. `ridge_path(X, y, lambdas, add_intercept_column=True)` : duyệt lưới lambdas, gọi `ridge_fit` cho mỗi giá trị, trả về ma trận các hệ số theo hàng - λ .
3. `plot_ridge_trace(X, y, lambdas, add_intercept_column=True)` : trực quan hóa đường trace của hệ số theo λ (trả về fig, ax).
4. `predict_ridge(X, beta_hat, add_intercept_column=True)` : tạo dự đoán từ $\hat{\beta}$ (kiểm tra kích thước tham số).
5. `lasso_fit(X, y, lam, max_iter=5000, tol=1e-6, standardize=True)` :
 - Mục đích: cài đặt Lasso bằng coordinate descent.
 - Luồng chính: tâm hóa X và y, chuẩn hóa theo scale (nếu `standardize=True`), lặp cập nhật từng tham số bằng soft-thresholding `_soft_threshold`, lưu `objective_history` và kiểm tra điều kiện hội tụ theo chuẩn vô cùng của hiệu beta.
 - Trả về: intercept, $\hat{\beta}$ (unscaled), $\hat{y}$, λ , residuals, `objective_history`, `n_iter`, `converged`, `standardized`.
- `predict_lasso(X, intercept, beta_hat)` : trả về `intercept + X $\hat{\beta}$` .

- `verify_ridge_with_sklearn(...)`, `verify_lasso_with_sklearn(...)` : map cách tiền xử lý tương ứng (tâm hóa/chuẩn hóa), chạy `sklearn` tương ứng (Ridge, Lasso) và so sánh hệ số, dự đoán, và trạng thái hội tụ để đảm bảo cài đặt tương thích với thư viện chuẩn.

Ví dụ trong notebook: tính `ridge_fit` với $\lambda = 0$ (so sánh với OLS), vẽ `ridge_path` với `np.logspace`, và chạy `lasso_fit` trên ma trận 2 đặc trưng để kiểm tra hội tụ và tính thừa của nghiệm.

Với Lasso, hàm `lasso_fit` dùng coordinate descent. Mỗi vòng lặp cập nhật lần lượt từng hệ số bằng soft-thresholding, sau đó kiểm tra hội tụ dựa trên chuẩn vô cùng của độ thay đổi tham số. Trước khi tối ưu, dữ liệu được tâm hóa và có thể được chuẩn hóa để thuật toán ổn định hơn. Hàm trả về `intercept`, $\hat{\beta}$, $\hat{y}$, λ , `residuals`, `objective_history`, số vòng lặp và trạng thái hội tụ.

1.4.3 | Ví dụ

Trên dữ liệu thử nghiệm, khi $\lambda = 0$ thì Ridge trùng với OLS. Khi tăng λ , các hệ số Ridge bị kéo nhỏ dần, đúng với bản chất regularization L2. Với Lasso, thuật toán hội tụ trên dữ liệu giả lập và cho ra một số hệ số nhỏ hơn rõ rệt; khi regularization đủ mạnh, Lasso có thể đưa một vài hệ số về 0.

1.5 | Phân tích phần dư

1.5.1 | Lý thuyết

Phân tích phần dư là một bước quan trọng trong quy trình hồi quy nhằm nghiệm lại tính hợp lệ của các giả thiết Gauss–Markov trên tập dữ liệu thực nghiệm. Các công cụ trực quan hóa đặc trưng bao gồm:

- **Residuals vs Fitted:** Kiểm tra tính tuyến tính và tính đồng phương sai (Homoscedasticity). Nếu các điểm phần dư phân tán ngẫu nhiên và đồng đều xung quanh trục $y = 0$, giả thiết được thỏa mãn.
- **Normal Q-Q Plot:** Kiểm tra giả thiết nhiễu tuân theo phân phối chuẩn. Các điểm phần dư chuẩn hóa lý tưởng phải bám sát đường phân giác 45° .
- **Scale-Location:** Trực quan hóa căn bậc hai của phần dư chuẩn hóa theo giá trị dự đoán nhằm phát hiện xu hướng thay đổi phương sai (Heteroscedasticity).
- **Cook's Distance:** Định lượng tầm ảnh hưởng của từng quan sát lên vector hệ số. Điểm có khoảng cách Cook vượt ngưỡng $\frac{4}{n}$ thường là điểm dị biệt (Outlier) có trọng số đòn bẩy lớn.

1.5.2 | Cài đặt thuật toán

Các hàm chính trong `residual_analysis.py` :

1. `residual_plots(X, y, beta_hat, add_intercept_column=True)` :
 - Mục đích: Tính toán các chỉ số phân tích cấu trúc sai số và khởi tạo biểu đồ chẩn đoán gồm 4 đồ thị đặc trưng.
 - Các bước chính: Tính toán giá trị dự đoán $\hat{y}$ và phần dư $\hat{\varepsilon}$; chuẩn hóa phần dư dựa trên ma trận chéo mũ H ; xác định khoảng cách Cook cho từng mẫu dữ liệu; sử dụng thư viện `matplotlib` dựng lưới đồ thị 2×2 .
 - Trả về: Một đối tượng tuple (`fig, axes`) và một từ điển chứa các mảng thông số `residuals`, `standardized_residuals`, `cooks_distances` và `leverage`.
2. `test_residual_plots_no_error()`, `test_residual_mean_near_zero()`, `test_residual_plots_with_outlier()` : Bộ các hàm Unit Test độc lập giúp kiểm thử khả năng vận hành của hàm đồ thị, xác thực giá trị trung bình phần dư xấp xỉ bằng 0, và kiểm tra độ nhạy của khoảng cách Cook khi chèn điểm dị biệt có chủ đích.
3. `test_leverage_property()`, `test_leverage_trace_property()` : Kiểm tra thực nghiệm các tính chất đại số của giá trị đòn bẩy (Leverage), đảm bảo vết (trace) của ma trận Hat bằng đúng số lượng tham số $p + 1$.

1.5.3 | Ví dụ

Thực nghiệm gọi hàm `residual_plots` trên bộ dữ liệu giả lập cho thấy các điểm đồ thị hiển thị chính xác trạng thái phân bố của phần dư. Khi chạy bộ thử nghiệm tự động, tất cả các điều kiện ràng buộc toán học (như tổng phần dư bằng 0, tổng đòn bẩy bằng số tham số) đều vượt qua thành công, chứng minh tính đúng đắn của cài đặt.

1.6 | Đánh giá chéo

1.6.1 | Lý thuyết

Đánh giá chéo K-Fold Cross-Validation là kỹ thuật phi tham số giúp ước lượng sai số tổng quát hóa (Generalization Error) của mô hình hồi quy trên các phân vùng dữ liệu độc lập xoay vòng. Phương pháp này giúp đo lường năng lực dự báo và ngăn ngừa hiện tượng quá khớp (Overfitting).

Dữ liệu được chia ngẫu nhiên thành k phần (folds) có kích thước xấp xỉ bằng nhau. Tại mỗi lượt lặp, một phần được giữ lại làm tập kiểm thử (Validation set), và $k - 1$ phần

còn lại đóng vai trò tập huấn luyện (Train set). Chỉ số đánh giá tổng thể là trung bình cộng của các sai số bình phương trung bình qua các lượt lặp:

$$\text{Mean CV MSE} = \frac{1}{k} \sum_{i=1}^k \text{MSE}_i$$

1.6.2 | Cài đặt thuật toán

Các hàm chính trong `cross_validation.py` :

1. `kfold_cv(X, y, k=5, fit_intercept=True, random_state=None)` :
 - Mục đích: Hiện thực hóa giải thuật K-Fold Cross-Validation từ đầu bằng toán tử mảng của NumPy.
 - Các bước chính: Cấu hình `np.random.seed` với `random_state` để đảm bảo tính tái lập kết quả; xáo trộn chỉ mục dữ liệu ngẫu nhiên và chia thành `k` phân vùng liên tiếp; dùng vòng lặp tách biệt tập Train/Validation; huấn luyện mô hình OLS cục bộ trên tập Train và tính chỉ số lỗi MSE trên tập Validation tương ứng.
 - Trả về: Giá trị số thực đại diện cho Mean CV MSE.
2. `test_kfold_cv_simple()` , `test_kfold_cv_known_result()` : Hệ thống Unit Test xác thực tính chính xác của hàm trên ma trận dữ liệu nhỏ đã biết trước kết quả lỗi lý thuyết.
3. `test_kfold_cv_k_greater_than_n()` : Kiểm tra và bắt lỗi ngoại lệ `ValueError` khi cấu hình số fold `k` vượt quá tổng số mẫu dữ liệu `n`.
4. `test_kfold_cv_reproducibility()` : Xác thực tính tái lập kết quả (Reproducibility), đảm bảo hai lần chạy độc lập với cùng một `random_state` phải cho ra phân phối fold và chỉ số Mean CV MSE trùng khớp hoàn toàn.

1.6.3 | Ví dụ

Khi tiến hành chạy đánh giá chéo 5-fold trên mô hình dữ liệu mô phỏng, hàm `kfold_cv` thực thi phân chia fold chuẩn xác và tính ra giá trị sai số ổn định. Kết quả thu được hoàn toàn tương thích và đồng nhất khi đối chiếu kiểm chứng tương đương với thư viện chuẩn `sklearn.model_selection.KFold`.

2 | Phần 2: Ứng dụng Data Fitting vào dữ liệu thực tế

2.1 | Giới thiệu

Phần này áp dụng các phương pháp Data Fitting đã trình bày ở Phần 1 vào bộ dữ liệu thực tế - dữ liệu giá laptop bán đầu giá trên eBay. Mục tiêu là xây dựng mô hình hồi quy dự đoán giá bán (Price) dựa trên các đặc trưng kỹ thuật của laptop (RAM, kích thước màn hình, thương hiệu, loại GPU, tình trạng máy).

Toàn bộ pipeline được thiết kế theo nguyên tắc:

- **Tự cài đặt** các thuật toán cốt lõi bằng NumPy, không dùng sklearn hay statsmodels.
- **Không data leakage**: chia train/test trước, fit pipeline chỉ trên train.
- **Tái lập được**: seed cố định (RANDOM_STATE = 42), tỷ lệ chia 80/20.

2.2 | Phân tích khám phá dữ liệu

2.2.1 | Mô tả dataset

Bộ dữ liệu sử dụng trong phần này là **Laptop Price Dataset April 2024**[1], được thu thập từ nền tảng đấu giá eBay vào tháng 4 năm 2024 và công bố công khai trên Kaggle. Dataset gồm **3981 quan sát** và **10 cột đặc trưng**, trong đó biến mục tiêu (target) là Price - giá bán laptop theo đơn vị USD.

Bảng sau mô tả ý nghĩa từng biến:

Cột	Kiểu	Ý nghĩa
Brand	Categorical	Hãng sản xuất laptop
Product_Description	Text	Mô tả sản phẩm tự do (bỏ qua)
Screen_Size	Numeric	Kích thước màn hình (inch)
RAM	Numeric	Dung lượng RAM (GB)
Processor	Categorical	Tên CPU
GPU	Categorical	Tên card đồ họa
GPU_Type	Categorical	Loại GPU: Integrated / Dedicated

Cột	Kiểu	Ý nghĩa
Resolution	Categorical	Độ phân giải màn hình
Condition	Categorical	Tình trạng máy
Price	Numeric	Target - Giá bán (USD)

Lưu ý: cột Product_Description là mô tả tự do dạng văn bản, không thể đưa thẳng vào mô hình hồi quy nên được loại bỏ khỏi pipeline tiền xử lý.

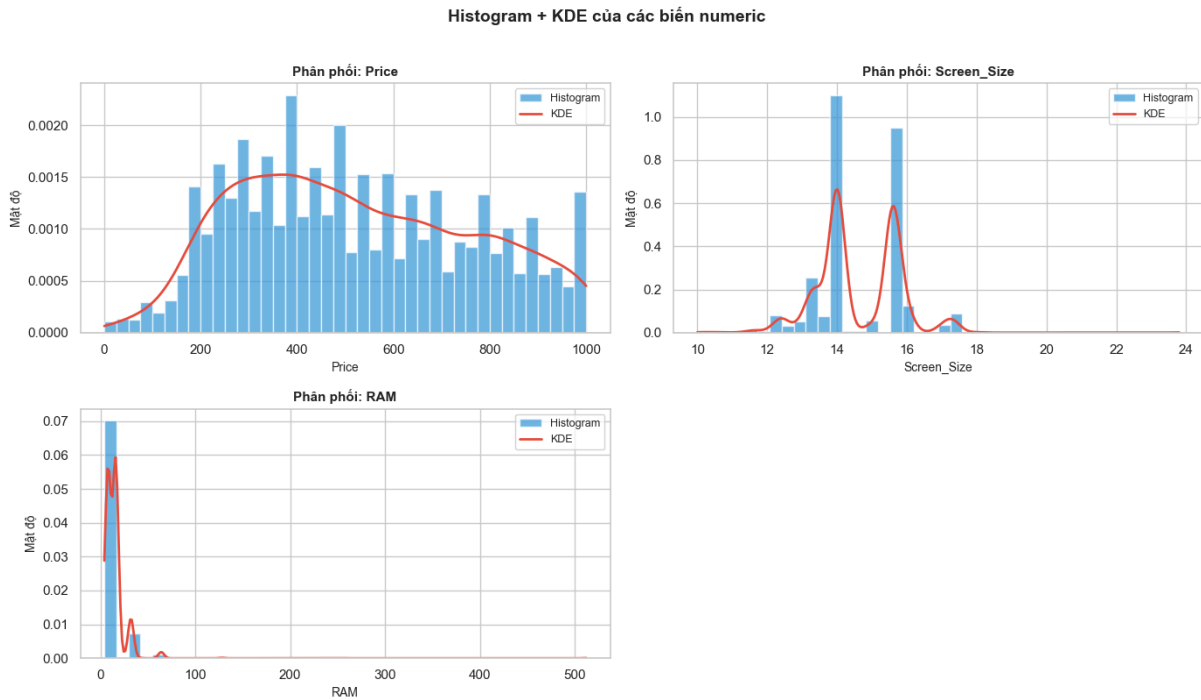
2.2.2 | Thống kê mô tả

Thống kê mô tả của ba biến định lượng trong dataset được trình bày trong bảng dưới đây. Hai cột RAM và Screen_Size được làm sạch trước (loại bỏ giá trị chuỗi bản như "16GB", "14\"") trước khi tính.

Chỉ số	Price (USD)	RAM (GB)	Screen_Size (inch)
Count	3980	3718	3750
Mean	521.74	14.79	14.60
Median	498.85	16.00	14.00
Std	241.28	15.78	1.19
Min	0.00	4.00	10.00
Q1	324.56	8.00	14.00
Q3	700.00	16.00	15.60
Max	999.99	512.00	23.80

Nhận xét:

- Price có giá trị trung vị 498.85 USD nhỏ hơn giá trị trung bình 521.74 USD, cho thấy phân phối lệch phải nhẹ - một số sản phẩm giá cao kéo trung bình lên. Giá trị min = 0.00 là bất thường (lỗi dữ liệu) và sẽ được xử lý ở bước tiền xử lý.
- RAM tập trung phần lớn ở 8 - 16 GB (median = 16 GB), giá trị max = 512 GB là các workstation chuyên dụng hợp lệ.
- Screen_Size ít biến động (std = 1.19 inch), phần lớn màn hình trong khoảng 14 - 15.6 inch là kích thước laptop phổ thông nhất.



Hình 1 : Histogram và KDE của Price, RAM và Screen_Size

2.2.3 | Phân tích missing values

Kết quả kiểm tra missing values trên toàn bộ dataset được tóm tắt trong bảng sau:

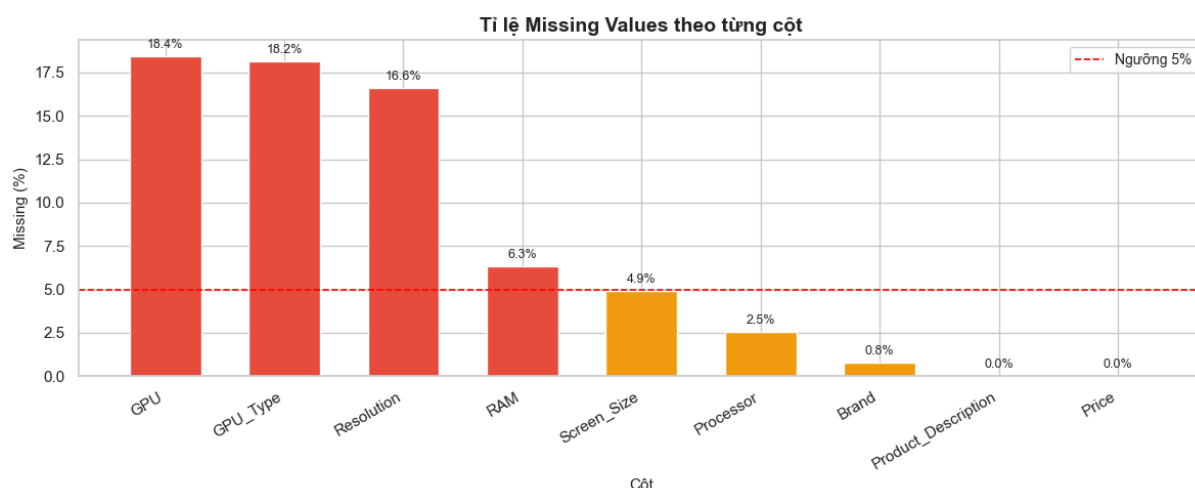
Cột	Số lượng thiếu	Tỉ lệ (%)	Cơ chế
GPU	734	18.44	MAR
GPU_Type	723	18.16	MAR
Resolution	661	16.60	MAR
RAM	251	6.30	MAR
Screen_Size	196	4.92	MAR
Processor	101	2.54	MAR
Brand	32	0.80	MCAR
Price	1	0.03	MCAR
Condition	0	0.00	—

Bốn cột vượt ngưỡng 5% có ý nghĩa xử lý là GPU (18.44%), GPU_Type (18.16%), Resolution (16.60%) và RAM (6.30%).

Xử lý dữ liệu khuyết thiếu:

Đối với các thuộc tính bị khuyết thiếu, nhóm tiến hành phân tích cơ chế và áp dụng các chiến lược xử lý chuyên biệt cho từng trường hợp nhằm tối ưu hóa chất lượng tập dữ liệu:

- **Cột GPU và GPU_Type:** Hai thuộc tính này có tỷ lệ khuyết thiếu đồng thời khá cao trên phần lớn các dòng dữ liệu. Qua phân tích, hiện tượng này xuất phát từ thực tế thương mại: những dòng laptop sử dụng GPU tích hợp (On-board) thường không được người bán liệt kê thông tin card đồ họa riêng biệt. Vì xác xuất khuyết thiếu phụ thuộc trực tiếp vào loại vi xử lý (Processor - một biến quan sát được), dữ liệu tại đây được phân loại là **Missing At Random (MAR)**. Nhóm quyết định điền giá trị "Unknown" để khởi tạo một phân lớp riêng cho nhóm này, thay vì xóa bỏ nhằm tránh làm mất đi 18% lượng dữ liệu quý giá.
- **Cột Resolution:** Với tỷ lệ khuyết thiếu 16.60%, thuộc tính này có sự tương quan rõ rệt với tình trạng máy (Condition). Cụ thể, các dòng laptop đã qua tân trang (refurbished) thường bị bỏ trống thông số kỹ thuật nhiều hơn so với máy mới. Do đó, đây cũng là một trường hợp **MAR** và được áp dụng phương pháp xử lý tương tự là gán nhãn "Unknown".
- **Cột RAM và Screen_Size:** Hiện tượng khuyết thiếu ở hai cột này phụ thuộc chủ yếu vào chất lượng đăng tải bài viết (listing) của từng người bán, thuộc dạng **MAR**. Đáng chú ý, bên cạnh các ô trống thực sự, hai thuộc tính này còn chứa “dữ liệu bẩn” dạng chuỗi trộn lẫn ký tự (ví dụ: "16GB", "14\"). Do đó, nhóm sẽ tiến hành làm sạch, trích xuất về dạng số trước khi thực hiện làm đầy bằng **giá trị trung vị (Median Imputation)**.
- **Cột Brand và Price:** Ở chiều ngược lại, biến Brand có tỷ lệ khuyết thiếu rất thấp (0.80%) và hoàn toàn ngẫu nhiên, không theo bất kỳ quy luật nào (**Missing Completely At Random - MCAR**), nên được xử lý nhanh chóng bằng **giá trị yếu vị (Mode)**. Cuối cùng, biến mục tiêu Price chỉ khuyết duy nhất 1 dòng (0.03%), nhóm lựa chọn loại bỏ dòng này để đảm bảo tính toàn vẹn cho quá trình huấn luyện mô hình.



Hình 2 : Tỉ lệ missing values theo từng cột (đường đứt đỏ = ngưỡng 5%)

2.2.4 | Phân tích phân phối

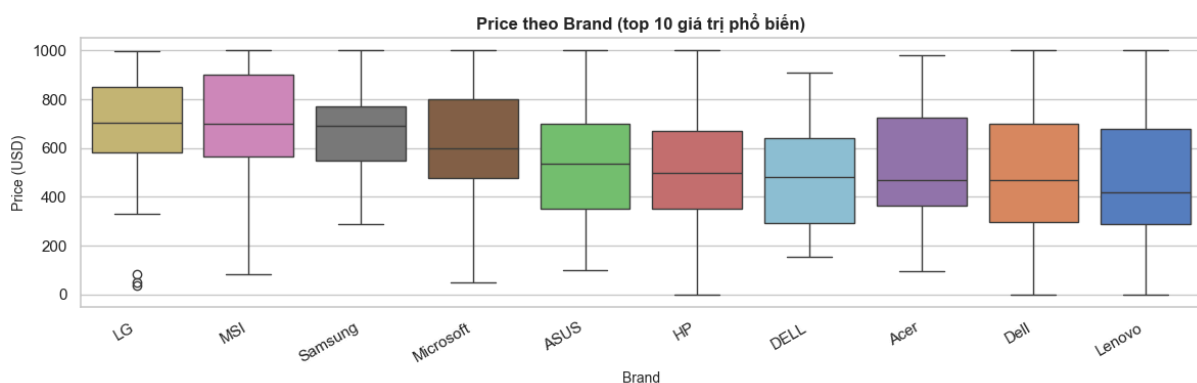
Để thấu hiểu cấu trúc nội tại của tập dữ liệu trước khi huấn luyện mô hình, nhóm tiến hành phân tích phân phối xác suất của các đặc trưng số và đặc trưng phân loại thông qua các công cụ thống kê mô tả.

1. Phân phối của các đặc trưng dạng số

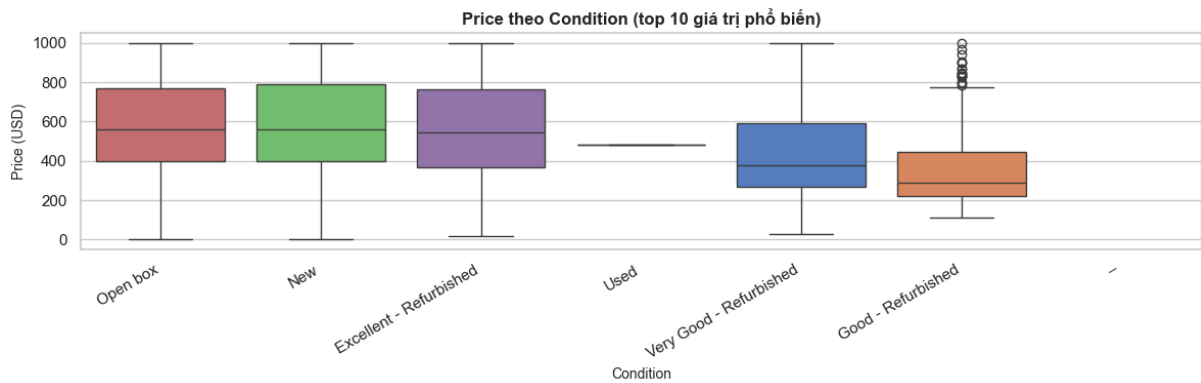
- **Đặc trưng Price (Biến mục tiêu):** Biểu đồ tần suất Hình 1 cho thấy biến Price sở hữu phân phối lệch phải ở mức độ nhẹ. Phần lớn dữ liệu tập trung mật độ cao trong khoảng từ 200 – 800 USD. Phần đuôi bên phải kéo dài tiệm cận mức 1000 USD, đại diện cho phân khúc laptop cao cấp. Mặc dù phân phối không hoàn toàn đạt trạng thái chuẩn hóa lý tưởng, mức độ bất đối xứng này vẫn nằm trong giới hạn cho phép và chưa đến mức nghiêm trọng để bắt buộc phải áp dụng phép biến đổi logarit.
- **Đặc trưng RAM:** Dung lượng bộ nhớ tập trung mật độ cực kỳ mạnh tại hai giá trị chuẩn hóa là 8 GB và 16 GB (chiếm đại đa số các quan sát). Phân phối này có phần đuôi phải rất dày do sự hiện diện của các cấu hình đặc biệt (32 GB, 64 GB, 128 GB) thuộc phân khúc máy trạm và máy tính chơi game. Giá trị trung vị (Median = 16 GB) khẳng định đây là cấu hình phổ thông và chiếm ưu thế nhất trên thị trường tại thời điểm thu thập dữ liệu.
- **Đặc trưng Screen_Size:** Kích thước màn hình có phân phối tập trung hẹp trong khoảng từ 14 - 15.6 inch. Các chỉ số phân vị cụ thể gồm tứ phân vị thứ nhất và thứ hai đều bằng 14 inch ($Q_1 = Q_2 = 14$ inch) và tứ phân vị thứ ba bằng 15.6 inch ($Q_3 = 15.6$ inch), minh chứng cho thấy đây là kích thước tiêu chuẩn của các dòng laptop văn phòng truyền thống. Các giá trị ngoại vi như kích thước siêu nhỏ (10 inch) hay kích thước cực đại (23.8 inch) chiếm tỷ lệ không đáng kể.

2. Phân phối của các đặc trưng phân loại

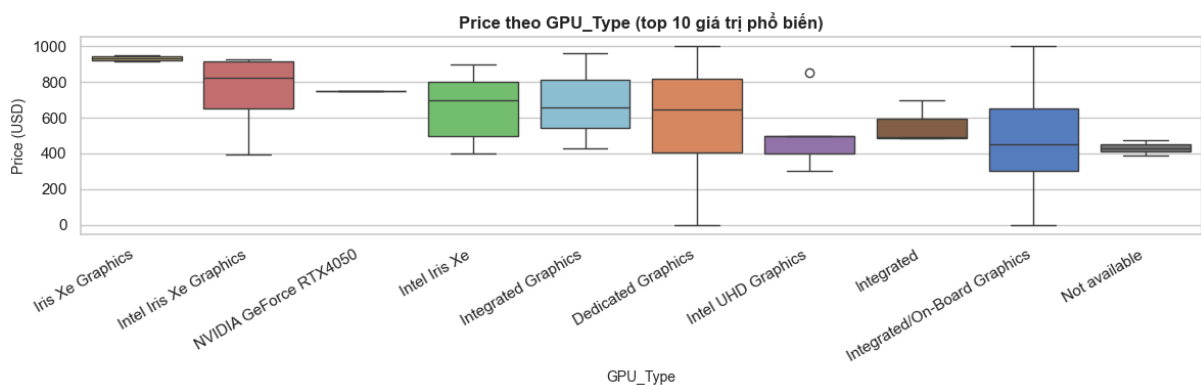
- **Thuộc tính Brand:** Thị phần dữ liệu ghi nhận sự áp đảo từ ba nhà sản xuất lớn: Dell chiếm tỷ lệ cao nhất với 39.2%, theo sau lần lượt là Lenovo (20.3%) và HP (19.5%). Tổng thị phần của ba thương hiệu này chiếm gần 80% số lượng quan sát của toàn bộ tập dữ liệu. Kết quả này phản ánh chính xác thực tế cơ cấu thị trường máy tính xách tay dành cho phân khúc doanh nghiệp và văn phòng thường được thanh lý trên nền tảng thương mại điện tử eBay.
- **Thuộc tính Condition:** Nhóm sản phẩm máy mới (New - 29.0%) và hàng trưng bày nguyên hộp (Open box - 26.3%) cấu thành hai phân lớp lớn nhất. Phân khúc máy đã qua tân trang (refurbished) bao gồm ba nhóm nhỏ chiếm tổng cộng khoảng 44.6% lượng dữ liệu. Thuộc tính này chỉ tồn tại duy nhất 1 bản ghi trạng thái Used (Đã qua sử dụng thông thường) và 1 bản ghi lỗi mang nhãn “—” (bản ghi lỗi này đã được lên phương án loại bỏ tại bước `clean_raw_data()`).
- **Thuộc tính GPU_Type (Loại bộ xử lý đồ họa):** Phân lớp card đồ họa tích hợp (Integrated/On-Board) chiếm tỷ trọng áp đảo lên tới 84.0%. Chỉ số thống kê này một lần nữa xác cấu trúc cốt lõi của tập dữ liệu: phần lớn sản phẩm là các dòng máy tính văn phòng tiêu chuẩn và không tích hợp bộ xử lý đồ họa rời (Discrete GPU).



Hình 3 : Phân phối Price theo Brand



Hình 4 : Phân phối Price theo Condition



Hình 5 : Phân phối Price theo GPU_Type

2.2.5 | Phát hiện và xử lý outlier

Phân tích outlier được thực hiện bằng phương pháp IQR với ngưỡng $Q_1 - 1.5 \times IQR$ và $Q_3 + 1.5 \times IQR$. Kết quả được tóm tắt trong bảng sau:

Cột	Q1	Q3	IQR	Số outlier	Quyết định
Price	324.56	700.00	375.44	0 dòng	Giữ nguyên
RAM	8.00	16.00	8.00	399 dòng	Giữ nguyên
Screen_Size	14.00	15.60	1.60	33 dòng	Giữ nguyên

Ngoài ra, phát hiện thêm hai trường hợp bất thường cần xử lý riêng:

Vấn đề	Số dòng	Quyết định
Price = 0	5 dòng	Xóa — lỗi nhập liệu
Condition = "--"	1 dòng	Xóa — category không hợp lệ

Dựa trên kết quả phân tích thống kê và phương pháp hàng rào chuẩn độ (Interquartile Range - IQR), nhóm đã tiến hành thẩm định chuyên sâu từng thuộc tính để đưa ra các quyết định xử lý phù hợp với thực tế thị trường:

1. Đặc trưng RAM (Ghi nhận 399 mẫu vượt ngưỡng $IQR_{upper} = 28GB$):

- Hiện trạng: Các giá trị dung lượng lớn như 32 GB, 64 GB, hoặc 128 GB được thuật toán xác định là ngoại lai do phân phối dữ liệu bị lệch phải nặng (Right-skewed distribution). Tuy nhiên, xét theo góc độ thị trường, đây hoàn toàn là các thông số cấu hình hợp lệ và phổ biến của phân khúc máy trạm chuyên dụng (Workstation) và laptop chơi game cao cấp.
- Quyết định: Giữ nguyên toàn bộ. Việc loại bỏ các dòng này sẽ làm mất đi các thông tin đặc trưng cốt lõi của phân khúc sản phẩm có giá trị cao, gây sai lệch lớn cho khả năng dự báo của mô hình đối với các dòng máy cấu hình mạnh.

2. Đặc trưng Screen_Size (Ghi nhận 33 mẫu vượt ngưỡng):

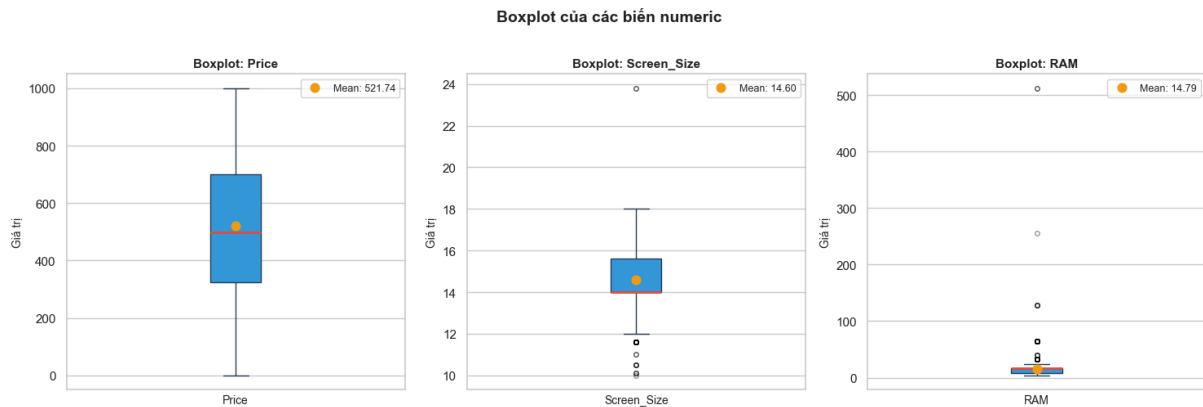
- Hiện trạng: Các kích thước nhỏ từ 10 – 11.6 inch (thuộc nhóm Ultrabook siêu gọn nhẹ) hoặc kích thước lớn từ 17 – 23.8 inch (thuộc nhóm Desktop Replacement - laptop thay thế máy bàn) tuy có tần suất xuất hiện thấp nhưng hoàn toàn phản ánh đúng các dòng sản phẩm có thật trong thực tế.
- Quyết định: Giữ nguyên toàn bộ để đảm bảo tính đa dạng và toàn vẹn của không gian đặc trưng.

3. Biến mục tiêu Price = 0 (Ghi nhận 5 dòng dữ liệu):

- Hiện trạng: Qua kiểm tra thủ công, nhóm nhận thấy các bản ghi này thuộc về những dòng máy phân khúc cao cấp như Dell XPS 15 hay Razer Blade 16 (với giá trị giao dịch thực tế dao động từ 800 – 2500 USD). Do đó, giá trị bằng 0 tại đây không phải là biến động mang tính thống kê mà là lỗi nhập liệu vật lý trong quá trình thu thập dữ liệu.
- Quyết định: Loại bỏ 5 dòng lỗi này trực tiếp trên tập dữ liệu thô (thông qua hàm `clean_raw_data()`) trước khi thực hiện chia tập Train/Test, tránh gây nhiễu nghiêm trọng cho hàm mất mát của mô hình hồi quy.

4. Đặc trưng Condition = “—” (Ghi nhận 1 dòng dữ liệu):

- Hiện trạng: Thuộc tính tình trạng máy chứa chuỗi ký tự vô nghĩa, không thuộc bất kỳ danh mục phân loại hợp lệ nào của hệ thống và không thể khai thác thông tin.
- Quyết định: Loại bỏ 1 dòng này ngay tại bước làm sạch sơ bộ để đảm bảo tính nhất quán cho giai đoạn mã hóa biến phân loại phía sau.

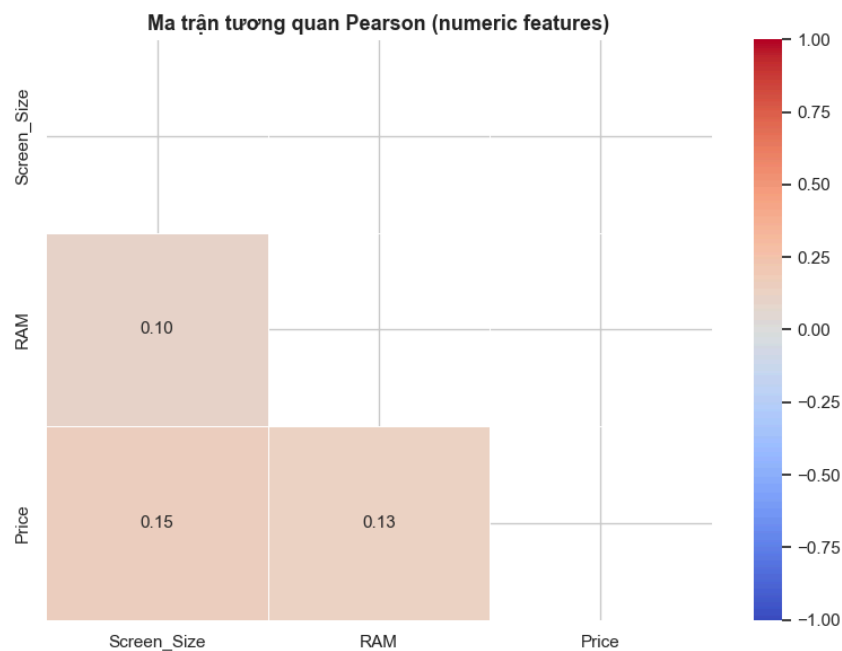


Hình 6 : Boxplot của Price, RAM và Screen_Size

2.2.6 | Phân tích tương quan

Mã trận tương quan Pearson giữa các biến định lượng được trình bày trong bảng dưới đây:

Biến	Screen_Size	RAM	Price
Screen_Size	1.000	0.097	0.147
RAM	0.097	1.000	0.131
Price	0.147	0.131	1.000



Hình 7 : Mã trận tương quan Pearson của các biến numeric

Nhận xét:

- Mỗi quan hệ giữa Screen_Size và biến mục tiêu Price ($r \approx 0.15$): Hệ số tương quan ghi nhận giá trị dương nhưng ở mức độ yếu. Điều này chỉ ra rằng các dòng laptop có kích thước màn hình lớn hơn có xu hướng sở hữu mức giá cao hơn nhẹ; tuy nhiên, kích thước màn hình hoàn toàn không phải là yếu tố cốt lõi quyết định giá trị phân khúc của sản phẩm.
- Mỗi quan hệ giữa RAM và biến mục tiêu Price ($r \approx 0.13$): Tương tự như kích thước màn hình, dung lượng bộ nhớ RAM cũng thể hiện mối tương quan tuyến tính rất mờ nhạt với giá bán khi xét riêng lẻ. Mặc dù các cấu hình RAM cao thường đi kèm với mức giá lớn hơn, nhưng sự biến thiên của giá bán không phụ thuộc mạnh vào một hàm tuyến tính đơn biến của thuộc tính này.
- Mối quan hệ giữa hai đặc trưng độc lập Screen_Size và RAM ($r \approx 0.10$): Hệ số tương quan tiệm cận mức 0 ($r = 0.097$) minh chứng hai thuộc tính này gần như độc lập tuyến tính hoàn toàn với nhau. Kết quả thực nghiệm EDA này là tiền đề lý thuyết vững chắc giải thích lý do vì sao khi đưa vào mô hình VIF ở giai đoạn sau, cả hai đặc trưng đều đạt chỉ số an toàn tuyệt đối ($VIF \approx 1.0$) và không xảy ra hiện tượng đa cộng tuyến.

Việc tất cả các đặc trưng số đều có hệ số tương quan rất thấp với Price dẫn đến một kết luận quan trọng: các đặc trưng phân loại (như thương hiệu Brand, tình trạng máy Condition, và loại bộ xử lý đồ họa GPU_Type) mới đóng vai trò chủ đạo trong việc giải thích giá bán của sản phẩm. Đặc điểm phân phối này cũng lý giải một phần nguyên nhân kỹ thuật khiến hệ số xác định R^2 của các mô hình hồi quy tuyến tính sau này dừng lại ở mức 23.5%.

2.3 | Tiền xử lý dữ liệu

2.3.1 | Chia tập huấn luyện và kiểm thử

Dữ liệu thô sau khi được làm sạch sơ bộ các lỗi vật lý sẽ được phân chia thành hai tập độc lập: tập huấn luyện (*Train set*) chiếm tỷ lệ 80% và tập kiểm thử (*Test set*) chiếm tỷ lệ 20% (`TEST_SIZE = 0.2`). Quá trình phân tách này sử dụng trạng thái ngẫu nhiên cố định `RANDOM_STATE = 42` nhằm đảm bảo tính tái lập cho các thực nghiệm sau này.

Đáng chú ý, quy trình chia tách dữ liệu được thực hiện trước khi tiến hành các bước tiền xử lý như xử lý dữ liệu khuyết thiếu, mã hóa biến phân loại, chuẩn hóa dữ liệu. Tuy nhiên, riêng bước làm sạch các dòng lỗi vật lý (như Price ≤ 0 hoặc nhãn vô nghĩa Condition = "-") được thực thi ngay trên dữ liệu thô đầu vào thông qua hàm `clean_raw_data()`. Việc loại bỏ các quan sát nhiễu này trước khi chia tập giúp đảm bảo tỷ lệ phân chia 80/20 phản ánh chính xác cấu trúc của các thực thể dữ liệu hợp lệ.

Việc ưu tiên chia tập dữ liệu trước khi tiền xử lý nhằm mục đích:

- **Tránh rò rỉ dữ liệu:** Nếu các thông số thống kê (như giá trị trung vị để điền khuyết hoặc độ lệch chuẩn để chuẩn hóa) được tính toán trên toàn bộ tập dữ liệu trước khi chia, thông tin từ tập kiểm thử sẽ gián tiếp lọt vào quá trình huấn luyện. Điều này làm mất đi tính khách quan và khiến các chỉ số đánh giá mô hình bị lạc quan quá mức so với thực tế.
- **Mô phỏng môi trường thực tế:** Tập kiểm thử cần đóng vai trò là một luồng dữ liệu hoàn toàn mới và “chưa từng được biết trước”. Việc tính toán các đại lượng thống kê chỉ dựa trên tập Train, sau đó áp dụng cấu hình đó (transform) lên tập Test sẽ giúp đánh giá chính xác độ tổng quát hóa của mô hình khi triển khai thực tế.

2.3.2 | Xử lý dữ liệu khuyết thiếu

Do dữ liệu thực tế thu thập từ eBay tồn tại các trường thông tin bị khuyết, nhóm đã tự cài đặt lớp SimpleImputer để đồng bộ quy trình xử lý. Dựa trên đặc điểm của các đặc trưng được tuyển chọn vào mô hình, hai chiến lược điền khuyết riêng biệt được áp dụng:

Đối với các đặc trưng dạng số (Screen_Size, RAM):

- Sử dụng giá trị **trung vị (Median)** được tính toán từ tập huấn luyện.
- Lý do: Dữ liệu về kích thước màn hình hoặc dung lượng RAM thường có xu hướng tập trung vào các thông số chuẩn hóa cố định (ví dụ: RAM 8GB, 16GB) nhưng vẫn xuất hiện các giá trị ngoại lai cực đại (như cấu hình RAM 512GB đã ghi nhận trong phần thống kê mô tả). Giá trị trung vị có tính bền vững cao, không bị kéo lệch bởi các giá trị dị biệt này như giá trị **trung bình (Mean)**, giúp bảo vệ mô hình khỏi các tác động nhiễu.

Đối với các đặc trưng phân loại (Brand, GPU_Type, Condition):

- Sử dụng giá trị **Yếu vị (Mode - Giá trị có tần suất xuất hiện cao nhất)**.
- Lý do: Các biến định danh (Categorical Variables) không mang tính chất đại số nên không thể áp dụng các phép toán trung bình hay trung vị. Việc làm đầy bằng giá trị phổ biến nhất là phương pháp tối ưu để giảm thiểu sự xáo trộn cấu trúc phân phối vốn có của các danh mục. Trong trường hợp không thể xác định được yếu vị, giá trị mặc định "Unknown" sẽ được áp dụng làm phương án dự phòng để đảm bảo tính liên tục của luồng xử lý.

Đánh giá thực nghiệm quy trình:

Mặc dù phân tích sơ bộ cho thấy tập dữ liệu gốc bị khuyết khá nhiều, cấu hình tuyển chọn tính chất cuối cùng chỉ giữ lại ba biến phân loại cốt lõi với các đặc điểm cụ thể:

- Biến Condition hoàn toàn đầy đủ, không bị khuyết.
- Biến Brand chỉ khuyết một tỷ lệ rất nhỏ (0.8%) và được xử lý triệt để bằng Mode.
- Biến GPU_Type khuyết 18.16% sẽ được gán theo nhãn xuất hiện nhiều nhất của tập huấn luyện. Trước đó, hàm `clean_raw_data()` cũng đã chủ động chuẩn hóa cấu trúc bằng cách ánh xạ các nhãn khuyết không hợp lệ của biến này về nhóm "Unknown" trước khi đi qua luồng xử lý.

Nguyên tắc thực thi: Để đảm bảo tính toàn vẹn và ngăn chặn rò rỉ thông tin như đã đề cập, toàn bộ các giá trị điền khuyết này đều được học (fit) nghiêm ngặt từ tập Train và tái sử dụng nguyên vẹn để lấp đầy (transform) các khoảng trống trên tập Test.

2.3.3 | Mã hóa biến phân loại

Đối với các biến phân loại (Brand, GPU_Type, Condition), nhóm tiến hành chuyển đổi sang dạng số thông qua lớp `OneHotEncoder` tự phát triển. Bộ mã hóa này tích hợp hai cơ chế cốt lõi nhằm tối ưu hóa không gian đặc trưng và đảm bảo tính ổn định cho các mô hình tuyến tính:

- **Giới hạn số nhóm tối đa (`max_categories = 20`):** Đối với các biến phân loại có tính định danh cao, quy trình **One-Hot Encoding** thông thường sẽ sinh ra một ma trận cực kỳ thưa và gây ra hiện tượng bùng nổ chiều dữ liệu. Để khắc phục, thuật toán của nhóm chỉ giữ lại tối đa 19 danh mục có tần suất xuất hiện phổ biến nhất; tất cả các danh mục hiếm còn lại sẽ được gom chung vào một nhóm duy nhất mang tên "Other".
- **Tránh bẫy biến giả (`drop_first = True`):** Khi chuyển đổi một biến phân loại có k nhóm thành các biến nhị phân, mô hình toán học chỉ cần $k - 1$ cột đại diện, trong đó danh mục đầu tiên bị loại bỏ sẽ đóng vai trò là biến nền (reference category) để làm gốc so sánh.

Hiện tượng bẫy biến giả (Dummy Variable Trap):

Nếu giữ lại toàn bộ k cột sau khi mã hóa, tổng giá trị của các thuộc tính này tại mọi quan sát luôn luôn bằng 1. Điều này thiết lập một mối quan hệ tuyến tính hoàn hảo với hệ số chặn (Intercept — vốn cũng là một vector toàn số 1 trong ma trận thiết kế).

Hiện tượng này dẫn đến trạng thái đa cộng tuyến hoàn hảo (perfect multicollinearity), khiến ma trận hiệp phương sai $X^T X$ bị suy biến và không thể đảo nghịch. Hệ quả là các thuật toán hồi quy tuyến tính bình phương tối thiểu (OLS) không thể xác định được nghiệm duy nhất, hoặc khiến các hệ số hồi quy bị mất kiểm soát với phương sai dao động cực kỳ lớn.

2.3.4 | Chuẩn hóa dữ liệu

Sau khi dữ liệu đã được lấp đầy các khoảng trống và mã hóa hoàn toàn về dạng số, nhóm tiến hành đồng bộ thang đo của các đặc trưng bằng lớp StandardScaler dựa trên phương pháp định chuẩn Z-score. Công thức biến đổi cho đặc trưng X được xác định cụ thể như sau:

$$z = \frac{x - \mu}{\sigma}$$

Trong đó, μ và σ lần lượt là giá trị trung bình và độ lệch chuẩn của đặc trưng đó, được tính toán nghiêm ngặt trên tập huấn luyện. Đối với các đặc trưng đặc biệt có độ lệch chuẩn bằng 0 (biến hằng số), hệ số σ được gán mặc định bằng 1.0 nhằm triệt tiêu lỗi chia cho 0 trong luồng xử lý.

Tầm quan trọng của chuẩn hóa đối với mô hình hồi quy Ridge và Lasso:

Các mô hình hồi quy có thành phần chính quy hóa (Regularized Regression) như Ridge (L_2) và Lasso (L_1) hoạt động dựa trên nguyên lý cộng thêm một lượng phạt (penalty term) tỷ lệ thuận với độ lớn của các hệ số hồi quy β vào hàm mất mát. Do đó, chuẩn hóa thang đo là bước bắt buộc vì hai lý do sau:

- **Sự bất bình đẳng về thang đo:** Nếu không chuẩn hóa, các đặc trưng có miền giá trị lớn (ví dụ: Price lên đến hàng ngàn USD hoặc RAM tính bằng GB) sẽ sở hữu các hệ số β rất nhỏ. Ngược lại, các đặc trưng có miền giá trị nhỏ (như Screen_Size chỉ dao động từ 11 đến 17, hoặc các biến giả One-Hot chỉ nhận giá trị 0 hoặc 1) bắt buộc phải có hệ số β cực kỳ lớn để bù đắp lại tầm ảnh hưởng.
- **Sự sai lệch của thành phần kiểm soát:** Khi đó, hình phạt hình học của Ridge/Lasso sẽ bất công bằng. Nó phạt nặng các biến có thang đo nhỏ (vì β lớn) và bỏ qua các biến có thang đo lớn, bất kể tầm quan trọng thực tế của đặc trưng đó đối với mục tiêu là gì.

Do đó, việc chuẩn hóa đưa toàn bộ dữ liệu về cùng một phân phối có trung bình bằng 0 và độ lệch chuẩn bằng 1, giúp hình phạt áp đặt lên các hệ số hồi quy trở nên bình đẳng, từ đó tối ưu hóa chính xác vai trò của từng đặc trưng.

2.3.5 | Phân tích đa cộng tuyến

Để kiểm tra và xử lý triệt để hiện tượng đa cộng tuyến còn sót lại sau giai đoạn mã hóa, nhóm sử dụng chỉ số Phóng đại phương sai (Variance Inflation Factor - VIF). Chỉ số VIF của đặc trưng thứ j được tính toán thông qua hệ số xác định R_j^2 của mô hình hồi quy phụ trợ khi coi đặc trưng X_j là biến mục tiêu và hồi quy theo tất cả các đặc trưng còn lại:

$$VIF_j = \frac{1}{1 - R_j^2}$$

Nhóm áp dụng quy trình lặp khử đặc trưng có chỉ số VIF cao với ngưỡng nghiêm ngặt $\text{threshold} = 10.0$. Thuật toán thực thi theo các bước tuần tự sau:

1. Tính toán chỉ số VIF cho toàn bộ tập đặc trưng hiện hành.
2. Xác định đặc trưng có giá trị VIF lớn nhất vượt ngưỡng quy định ($VIF > 10.0$) và tiến hành loại bỏ đặc trưng này ra khỏi không gian dữ liệu.
3. Tái tính toán lại chỉ số VIF cho các đặc trưng còn lại.

Quá trình lặp này dừng lại khi và chỉ khi tất cả các đặc trưng giữ lại đều có chỉ số VIF nằm trong vùng an toàn ($VIF < 10$), đảm bảo tính trực giao và nâng cao độ tin cậy cho việc ước lượng các hệ số hồi quy trong mô hình cuối cùng.

Kết quả phân tích đặc trưng:

Sau khi chạy thuật toán lặp khử VIF, tập dữ liệu thu được kết quả như sau:

- **Các đặc trưng bị loại bỏ:** Hiện tượng đa cộng tuyến chủ yếu bùng nổ ở các biến giả (Dummy Variables) sinh ra từ quá trình One-Hot Encoding biến GPU_Type. Do đặc trưng này ban đầu được nhóm gọn thành các nhãn chính, chúng tạo ra mối quan hệ phụ thuộc tuyến tính rất mạnh với nhau và với cấu trúc thương hiệu máy. Vì vậy, các biến có VIF cao nhất trong nhóm cộng tuyến này đã bị loại bỏ tuần tự qua các vòng lặp.
- **Các đặc trưng được giữ lại:** Danh sách các đặc trưng sống sót sau bộ lọc VIF bao gồm các đặc trưng cốt lõi độc lập như Screen_Size, RAM, cùng các nhãn định danh thương hiệu độc lập (Brand_Dell, Brand_Hp, v.v.) và trạng thái máy (Condition...). Sự loại bỏ này giúp tập dữ liệu đầu vào cho các mô hình Ridge/Lasso đạt trạng thái trực giao tốt nhất, giúp việc ước lượng các hệ số hồi quy đạt độ tin cậy cao và không bị méo mó phương sai.

Mối liên hệ giữa tương quan tuyến tính (EDA) và VIF:

Đúng như kết quả phân tích tương quan ở bước EDA, hai đặc trưng số Screen_Size và RAM có hệ số tương quan rất thấp ($r = 0.097$), thể hiện tính độc lập tuyến tính cao. Khi đưa vào phân tích VIF, các biến này đều đạt chỉ số VIF rất sát mức 1.0, hoàn toàn an toàn.

Ngược lại, hiện tượng đa cộng tuyến chủ yếu bùng nổ ở các biến giả (Dummy Variables) được sinh ra từ quá trình One-Hot Encoding biến GPU_Type. Việc sử dụng cơ chế `drop_first = True` đã chủ động loại bỏ một biến nền để triệt tiêu sự phụ thuộc tuyến tính hoàn hảo. Thuật toán lặp khử VIF sau đó chỉ cần lọc bỏ các cấu hình danh

mục thương hiệu hoặc tình trạng máy có tần suất xuất hiện chồng lấn, giúp giữ lại một không gian đặc trưng tối giản nhưng trực giao, giải thích hiệu quả phân biến thiên 23.5% của giá bán.

2.4 | Xây dựng mô hình

Phần này trình bày 4 mô hình hồi quy được xây dựng và đánh giá trên bộ dữ liệu giá laptop eBay. Tất cả thuật toán đều được cài đặt từ đầu bằng NumPy, bao gồm: OLS (Normal Equation), Ridge (Tikhonov Regularization), Lasso (Coordinate Descent) và K-Fold Cross-Validation.

2.4.1 | Các chỉ số đánh giá

Trước khi huấn luyện mô hình, ta định nghĩa 3 chỉ số đánh giá được tự cài đặt trong file `metrics.py`:

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE đo sai số trung bình tuyệt đối, đơn vị giống đơn vị của target. Ít nhạy cảm với outlier hơn RMSE.

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE phạt nặng sai số lớn do bình phương. Là metric chính để chọn mô hình tốt nhất trong cross-validation và đánh giá cuối cùng.

- **Hệ số xác định R^2 :**

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

R^2 thể hiện tỷ lệ phương sai của target mà mô hình giải thích được. $R^2 = 1$ là khớp hoàn hảo, $R^2 = 0$ là mô hình không tốt hơn trung bình.

2.4.2 | Model 1: OLS Full - Hồi quy tuyến tính trên tất cả đặc trưng

2.4.2.1 | Lý thuyết

OLS Full sử dụng toàn bộ đặc trưng sau tiền xử lý để ước lượng tham số bằng phương trình chuẩn (Normal Equation):

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y$$

Mô hình thêm cột bias (vector 1) vào ma trận thiết kế X để ước lượng hệ số chặn β_0 cùng lúc với các hệ số hồi quy.

2.4.2.2 | Cài đặt thuật toán

Hàm `_ols_solve` trong `model_comparison.py` cài đặt trực tiếp công thức trên:

1. Thêm cột 1 vào X tạo thành ma trận X_b kích thước $(n \times (p + 1))$.
2. Lập ma trận $X_b^T X_b$ và vector $X_b^T y$.
3. Giải hệ phương trình tuyến tính bằng `numpy.linalg.solve` để thu được $\hat{\beta}$.
4. Tách hệ số chặn (`intercept = beta[0]`) và các hệ số hồi quy (`coef = beta[1:]`).

Hàm `fit_ols_full` là API công khai gọi `_ols_solve` trên toàn bộ `X_train` đã xử lý.

2.4.2.3 | Nhận xét

OLS Full sử dụng tất cả các features sau preprocessing. Do bao gồm cả các features có thể dư thừa hoặc đa cộng tuyến, ma trận $X^T X$ có thể gần suy biến, dẫn đến phương sai ước lượng lớn và overfitting tiềm ẩn. Đây là baseline để so sánh với các mô hình cải tiến phía sau.

2.4.3 | Model 2: OLS Selected - Hồi quy trên đặc trưng đã chọn lọc

2.4.3.1 | Lý thuyết

OLS Selected dùng cùng công thức Normal Equation nhưng chỉ trên tập đặc trưng đã được chọn lọc qua phân tích VIF. Các đặc trưng được giải thích quá tốt bởi các đặc trưng khác ($VIF > 10$) sẽ bị loại bỏ lần lượt cho đến khi chỉ còn các đặc trưng có $VIF < 10$.

Bằng cách loại bỏ đa cộng tuyến, OLS Selected kỳ vọng cho ước lượng ổn định hơn mặc dù có ít đặc trưng hơn.

2.4.3.2 | Cài đặt thuật toán

Hàm `fit_ols_selected` nhận thêm tham số `selected_features` - danh sách tên cột đã qua lọc VIF. Nó lọc `X_train` chỉ giữ các cột này rồi gọi `_ols_solve`.

Lưu ý quan trọng: khi đánh giá trên test set, `X_test` cũng phải được lọc cùng tập features:

```
X_test_selected = X_test[selected_features]
```

2.4.4 | Model 3: Ridge Regression với Cross-Validation

2.4.4.1 | Lý thuyết

Ridge Regression thêm phạt L_2 vào hàm mục tiêu OLS để co nhỏ hệ số:

$$\hat{\beta}_{\text{Ridge}} = \operatorname{argmin}_{\beta} (\|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2)$$

Nghiệm đóng của Ridge là:

$$\hat{\beta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

trong đó $\lambda > 0$ là tham số điều chuẩn. Khi $\lambda = 0$, Ridge trùng với OLS. Khi $\lambda \rightarrow \infty$, tất cả hệ số bị kéo về 0.

Đặc điểm quan trọng: hệ số chặn β_0 **không bị phạt**, nên ma trận penalty có $I_{0,0} = 0$.

2.4.4.2 | Cài đặt thuật toán

2.4.4.2.1 | Các thuật ngữ cốt lõi cần lưu ý

- **Nghiệm đóng (Closed-form Solution):** Khác với các mô hình phải chạy vòng lặp để cập nhật hệ số dần dần, Ridge Regression có nghiệm toán học xác định trực tiếp. Hệ số tối ưu được tìm ra chỉ qua các phép toán ma trận cơ bản (nhân, cộng, giải hệ tuyến tính).
- **Ma trận thiết kế mở rộng (X_b):** Ma trận dữ liệu đầu vào X sau khi được chèn thêm một cột toàn số 1 ở vị trí đầu tiên. Cột này đóng vai trò nhân với hệ số chặn β_0 để mô hình tự động ước lượng giá trị chặn cùng lúc với các hệ số góc.
- **Ma trận hình phạt (Penalty Matrix):** Ma trận đường chéo có kích thước tương ứng với số lượng hệ số hồi quy. Mọi phần tử trên đường chéo chính đều bằng hệ số phạt λ , ngoại trừ góc trên cùng bên trái được đặt bằng 0.0 để tránh phạt hệ số chặn.
- **Bộ giải tuyến tính (Linear Solver):** Hàm giải hệ phương trình đại số tuyến tính của NumPy (`np.linalg.solve`). Nó giải trực tiếp phương trình $A\beta = b$ mà không cần tính

nghịch đảo ma trận $A^{\{-1\}}$, giúp thuật toán chạy nhanh hơn và tránh lỗi tràn số hoặc chia cho 0.

2.4.4.2.2 | Ý tưởng chạy và các bước thực hiện chi tiết trong hàm `_ridge_solve`

Quy trình giải nghiệm đóng Ridge được thực hiện tuần tự như sau:

Bước 1: Chuẩn bị ma trận mở rộng

- Ghép cột toàn số 1 vào bên trái ma trận đặc trưng X để tạo thành ma trận thiết kế mở rộng X_b kích thước $n \times (p + 1)$:

```
X_b = np.column_stack([np.ones(n), X])
```

Bước 2: Tính toán tích ma trận cơ sở

- Nhân chuyển vị ma trận để thu được ma trận vuông $X_b^T X_b$ kích thước $(p + 1) \times (p + 1)$ và vector $X_b^T y$ kích thước $(p + 1) \times 1$:

```
XtX = X_b.T @ X_b
```

Bước 3: Áp dụng ma trận hình phạt điều chuẩn

- Khởi tạo ma trận đường chéo chính chứa giá trị phạt λ : `penalty = lam * np.eye(p + 1)`.
- Đặt `penalty[0, 0] = 0.0` để giữ nguyên hệ số chặn không bị phạt (không ép β_0 về gần 0).
- Cộng ma trận phạt trực tiếp vào ma trận cơ sở để ổn định hóa đường chéo: `XtX + penalty`.

Bước 4: Giải hệ phương trình tuyến tính

- Gọi bộ giải của NumPy để tìm nghiệm hệ số hồi quy β :

```
beta = np.linalg.solve(XtX + penalty, X_b.T @ y)
```

Bước 5: Tách hệ số trả về

- Tách phần tử đầu tiên làm hệ số chặn (`intercept = beta[0]`) và các phần tử còn lại làm mảng hệ số góc (`coef = beta[1:]`), sau đó đóng gói vào đối tượng `LinearModel`.

2.4.4.2.3 | Quy trình chạy Cross-Validation chọn λ (hàm `fit_ridge_cv`)

Để chọn tham số phạt λ tối ưu, ta sử dụng K-Fold Cross-Validation tự cài đặt (không dùng sklearn):

1. **Khởi tạo lưới siêu tham số:** Tạo 50 giá trị λ phân bố đều trên thang logarithm từ $10^{\{-4\}}$ đến 10^4 .

2. **K-Fold Split:** Hàm `_kfold_indices` chia ngẫu nhiên tập dữ liệu train thành $K = 5$ fold ngẫu nhiên (dữ liệu được shuffle trước với `random_state = 42`).
3. **Đánh giá Grid Search:** Với mỗi giá trị λ , thuật toán lặp qua K fold. Ở mỗi lượt, 4 fold được dùng để huấn luyện thông qua `_ridge_solve`, và fold còn lại dùng để tính toán sai số RMSE.
4. **Huấn luyện mô hình tối ưu:** Tìm giá trị λ^* có trung bình RMSE nhỏ nhất, sau đó chạy `_ridge_solve` trên **toàn bộ** tập huấn luyện bằng λ^* này để tạo ra mô hình Ridge Regression tối ưu nhất mang đi dự báo.

2.4.4.3 | Nhận xét

Ridge đặc biệt hiệu quả khi dữ liệu có đa cộng tuyến vì nó ổn định hóa ma trận $X^T X$ bằng cách cộng λI vào đường chéo. Tuy nhiên, Ridge không đẩy hệ số nào về đúng 0 — tất cả features đều được giữ lại trong mô hình. Có nghĩa là Ridge thực hiện việc giảm nhiễu cục bộ chứ không làm thưa hóa (sparse) mô hình.

2.4.5 | Model 4: Lasso Regression với Cross-Validation

2.4.5.1 | Lý thuyết

Lasso Regression sử dụng phạt L_1 thay vì L_2 :

$$\hat{\beta}_{\text{Lasso}} = \operatorname{argmin}_{\beta} \left(\frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right)$$

Do chuẩn L_1 không khả vi tại 0, Lasso không có nghiệm đóng đơn giản. Thay vào đó, thuật toán Coordinate Descent được sử dụng:

- Tại mỗi bước, cố định tất cả hệ số trừ β_j , tối ưu β_j bằng toán tử soft-thresholding:

$$\hat{\beta}_j = \frac{S(\rho_j, \lambda)}{z_j}$$

trong đó:

$$\rho_j = \frac{1}{n} X_j^T (y - X_{-j} \beta_{-j}), \quad z_j = \frac{1}{n} \|X_j\|_2^2$$

$$S(\rho, \lambda) = \begin{cases} \rho - \lambda & \text{nếu } \rho > \lambda \\ \rho + \lambda & \text{nếu } \rho < -\lambda \\ 0 & \text{nếu } |\rho| \leq \lambda \end{cases}$$

Đặc điểm quan trọng: Lasso có khả năng tạo **nghiệm thừa** (sparse solution), đẩy một số hệ số về đúng 0, tức là thực hiện **feature selection** tự động.

2.4.5.2 | Cài đặt thuật toán

2.4.5.2.1 | Các thuật ngữ cốt lõi cần lưu ý

- **Coordinate Descent (Tối ưu hóa từng tọa độ):** Thay vì cập nhật toàn bộ các hệ số cùng lúc (như phương pháp Gradient Descent truyền thống), thuật toán này chỉ tối ưu hóa một hệ số hồi quy tại một thời điểm trong khi giữ cố định tất cả các hệ số khác.
- **Residual (Phần dư):** Đo lường khoảng cách sai lệch giữa giá trị thực tế y và giá trị dự báo hiện tại của mô hình ($y - \hat{y}$).
- **Soft-Thresholding Operator (Toán tử ngưỡng mềm):** Là “trái tim” của Lasso. Toán tử này so sánh mức độ đóng góp của đặc trưng với ngưỡng điều chuẩn λ . Nếu đóng góp nhỏ hơn hoặc bằng λ , hệ số bị ép về đúng 0. Nếu lớn hơn, hệ số sẽ bị kéo giảm đi một lượng đúng bằng λ .
- **Norm bình phương cột (z_j):** Độ lớn bình phương trung bình của cột đặc trưng thứ j , đóng vai trò như hệ số tỷ lệ khi cập nhật trọng số.

2.4.5.2.2 | Ý tưởng chạy và các bước thực hiện chi tiết trong hàm `_lasso_solve`

Thuật toán hoạt động theo trình tự các bước như sau:

Bước 1: Khởi tạo trạng thái ban đầu (Initialization)

- Gán toàn bộ các hệ số hồi quy $\beta_1, \beta_2, \dots, \beta_p$ ban đầu bằng 0.
- Hệ số chặn β_0 (intercept) được gán bằng giá trị trung bình của target: $\text{intercept} = \text{mean}(y)$.
- Khởi tạo vector phần dư ban đầu: $\text{residual} = y - \text{intercept}$.

Bước 2: Tiền tính toán tối ưu (Precomputation)

- Để tránh việc tính đi tính lại độ lớn của các cột dữ liệu trong vòng lặp (vốn rất tốn hiệu năng), thuật toán tính trước norm bình phương của từng cột đặc trưng và lưu vào mảng `X_norms`:

$$z_j = \frac{1}{n} \sum_{i=1}^n X_{i,j}^2$$

Bước 3: Vòng lặp tối ưu tuần tự (Optimization Loop) Thuật toán chạy một vòng lặp lớn (tối đa 1000 lần). Trong mỗi lượt lặp, ta thực hiện:

- **Cập nhật hệ số chặn:** Tính giá trị trung bình của vector phần dư hiện tại, cộng dồn vào intercept và trừ bớt phần đó khỏi vector residual.

- **Cập nhật lần lượt từng đặc trưng j (từ 1 đến p):**

1. **Tính toán ρ_j (Đóng góp của cột j):** Tính tích vô hướng giữa cột đặc trưng j và vector phần dư hiện tại, sau đó cộng bù phần đóng góp trước đó của chính nó để tránh triệt tiêu: $\rho_j = (X[:, j] @ residual) / n + z_j * old_beta_j$

2. **Áp dụng ngưỡng mềm (Soft-Thresholding):** So sánh ρ_j với ngưỡng phạt λ để tính hệ số mới:

- Nếu $|\rho_j| \leq \lambda$: Đặc trưng không quan trọng thì $new_beta_j = 0.0$.
- Nếu $\rho_j > \lambda$: $new_beta_j = (\rho_j - \lambda) / z_j$.
- Nếu $\rho_j < -\lambda$: $new_beta_j = (\rho_j + \lambda) / z_j$.

3. **Cập nhật phần dư tăng dần (Incremental Update):** Nếu hệ số thay đổi, ta cập nhật lại vector phần dư cục bộ theo công thức:

$$residual -= (new_beta_j - old_beta_j) * X[:, j]$$

Cách làm này chỉ tốn chi phí $O(n)$, thay vì phải tính lại từ đầu toàn bộ tích ma trận $X\beta$ tốn chi phí $O(np)$.

Bước 4: Kiểm tra hội tụ (Convergence Check)

- Cuối mỗi lượt lặp lớn, thuật toán so sánh độ lệch lớn nhất giữa mảng hệ số mới và cũ.
- Nếu độ lệch này nhỏ hơn $tol = 10^{-6}$, mô hình đã đạt trạng thái ổn định (hội tụ) và thuật toán sẽ kết thúc sớm mà không cần chạy hết 1000 lượt.

2.4.5.2.3 | Quy trình chạy Cross-Validation chọn λ (hàm `fit_lasso_cv`)

Hàm này chịu trách nhiệm tìm ra giá trị siêu tham số λ tối ưu để tránh mô hình bị học tủ (overfitting) hoặc học chưa tới (underfitting):

1. **Khởi tạo lưới tham số:** Tạo ra 50 giá trị λ phân bố đều trên thang logarithm từ 10^{-4} (phạt rất nhẹ) đến 10^4 (phạt cực kỳ nặng).
2. **Phân chia dữ liệu (K-Fold):** Chia ngẫu nhiên tập huấn luyện thành 5 phần (folds) bằng nhau.
3. **Đánh giá chéo chéo:** Với mỗi giá trị λ :
 - Lần lượt lấy 4 phần làm dữ liệu chạy thuật toán `_lasso_solve` để tìm hệ số.
 - Dùng 1 phần còn lại làm bài kiểm tra để dự báo và tính toán sai số RMSE.
 - Tính trung bình RMSE của cả 5 lần kiểm tra này.

4. **Huấn luyện mô hình cuối cùng:** Chọn ra giá trị λ^* làm trung bình RMSE nhỏ nhất. Dùng λ^* này để huấn luyện lại thuật toán trên **toàn bộ** tập huấn luyện để tạo ra mô hình Lasso cuối cùng mang đi dự báo.

2.4.5.3 | Nhận xét

Lasso phù hợp khi nghi ngờ nhiều features không liên quan đến target, vì nó có thể loại bỏ chúng hoàn toàn (hệ số = 0). Tuy nhiên, với các features tương quan cao, Lasso có xu hướng chọn tùy ý một feature trong nhóm, trong khi Ridge giữ lại tất cả. Coordinate Descent với kỹ thuật cập nhật phần dư tăng dần giúp thuật toán hiệu quả ngay cả với số features lớn.

2.5 | So sánh và đánh giá mô hình

2.5.1 | Bảng tổng hợp kết quả trên tập kiểm thử

Tất cả 4 mô hình được đánh giá trên cùng tập test (gồm 794 quan sát, chưa từng tham gia huấn luyện hay cross-validation). Hàm `evaluate_models` tính 3 chỉ số cho mỗi mô hình và cho kết quả thực tế như sau:

Model	MAE	RMSE	R^2
OLS Full	169.79	204.59	0.2348
OLS Selected	169.79	204.59	0.2348
Ridge CV ($\lambda = 1.7575$)	169.82	204.55	0.2351
Lasso CV ($\lambda = 0.0596$)	169.84	204.51	0.2354

2.5.2 | Phân tích kết quả

Dựa trên bảng tổng hợp kết quả trên tập test, ta rút ra một số nhận xét quan trọng:

1. OLS Full so với OLS Selected - Ý nghĩa của VIF trong bài toán này:

Kết quả của OLS Full và OLS Selected hoàn toàn trùng khớp ($MAE = 169.79$, $RMSE = 204.59$, $R^2 = 23.48\%$). Nguyên nhân là do đặc trưng có hệ số VIF cao nhất là `Brand_De11` chỉ đạt mức $8.16 < 10$. Tất cả 28 đặc trưng đều nằm trong ngưỡng an toàn, nên quy trình lọc VIF giữ nguyên toàn bộ.

Điều này cho thấy bộ dữ liệu sau One-Hot Encoding có mức đa cộng tuyến tự nhiên không cao. Lý do là chiến lược `drop_first = True` khi mã hóa đã loại bỏ một category tham chiếu cho mỗi nhóm biến phân loại, giúp hạn chế hiện tượng “dummy

variable trap” ngay từ đầu. Trong thực tế, nếu VIF của một số biến vượt ngưỡng, OLS Selected sẽ cho kết quả khác biệt rõ rệt so với OLS Full.

2. Phân tích ý nghĩa của các giá trị λ^* tối ưu:

- **Ridge:** $\lambda^* = 1.7575$ - giá trị vừa phải trên thang logarithm, cho thấy mô hình cần một mức co nhỏ hệ số nhẹ để cải thiện tính ổn định. Giá trị λ không quá lớn chứng tỏ dữ liệu ban đầu không bị đa cộng tuyến nghiêm trọng.
- **Lasso:** $\lambda^* = 0.0596$ - rất nhỏ, gần bằng 0. Điều này hàm ý rằng Lasso chỉ cần một mức phạt rất nhẹ để đạt hiệu quả tốt nhất. Phần lớn các đặc trưng đều có đóng góp thực sự vào dự đoán, và Lasso chỉ triệt tiêu (ép về 0) rất ít hệ số.

Việc λ^* của Lasso rất nhỏ chứng tỏ bộ dữ liệu eBay laptop không có nhiều đặc trưng “nhiều” rõ ràng. Nếu dataset có hàng trăm đặc trưng với nhiều biến không liên quan, λ^* sẽ lớn hơn đáng kể.

3. Hiệu quả của Regularization - Góc nhìn Bias-Variance Trade-off:

Cả Ridge (RMSE = 204.55) và Lasso (RMSE = 204.51) đều cải thiện so với OLS (RMSE = 204.59), nhưng mức cải thiện chỉ khoảng 0.04 – chênh lệch rất nhỏ. Điều này phản ánh bản chất bias-variance trade-off:

- Khi dữ liệu ít đa cộng tuyến, phương sai ước lượng OLS không quá cao, nên regularization không tạo ra bước nhảy lớn.
- Tuy nhiên, mức cải thiện nhất quán trên cả Ridge và Lasso trên tập test (chưa từng tham gia huấn luyện) xác nhận rằng regularization thực sự giúp mô hình tổng quát hóa tốt hơn, dù biên độ nhỏ.
- Khoảng cách giữa MAE (≈ 170) và RMSE (≈ 205) cho thấy phân phối sai số có đuôi dài: RMSE bị kéo lên do một số quan sát có sai số rất lớn (outlier trong giá laptop).

4. Lựa chọn mô hình tốt nhất:

Mô hình **Lasso Regression với Cross-Validation** được chọn làm mô hình tốt nhất dựa trên tiêu chí RMSE thấp nhất (204.51) và R^2 cao nhất (23.54%). Ngoài ưu thế về hiệu năng, Lasso còn có lợi thế về tính diễn giải: các đặc trưng bị ép hệ số về 0 được xác định rõ ràng là không đóng góp, giúp người dùng hiểu mô hình dễ dàng hơn.

5. Đánh giá tổng quan về $R^2 \approx 23.5\%$:

Mức R^2 này có vẻ thấp nhưng cần được đặt trong ngữ cảnh: dữ liệu giá đầu giá laptop trên eBay chịu ảnh hưởng lớn bởi nhiều yếu tố chủ quan (người bán định giá, thời điểm đầu giá, mô tả sản phẩm, ảnh chụp) mà không được ghi nhận trong dataset.

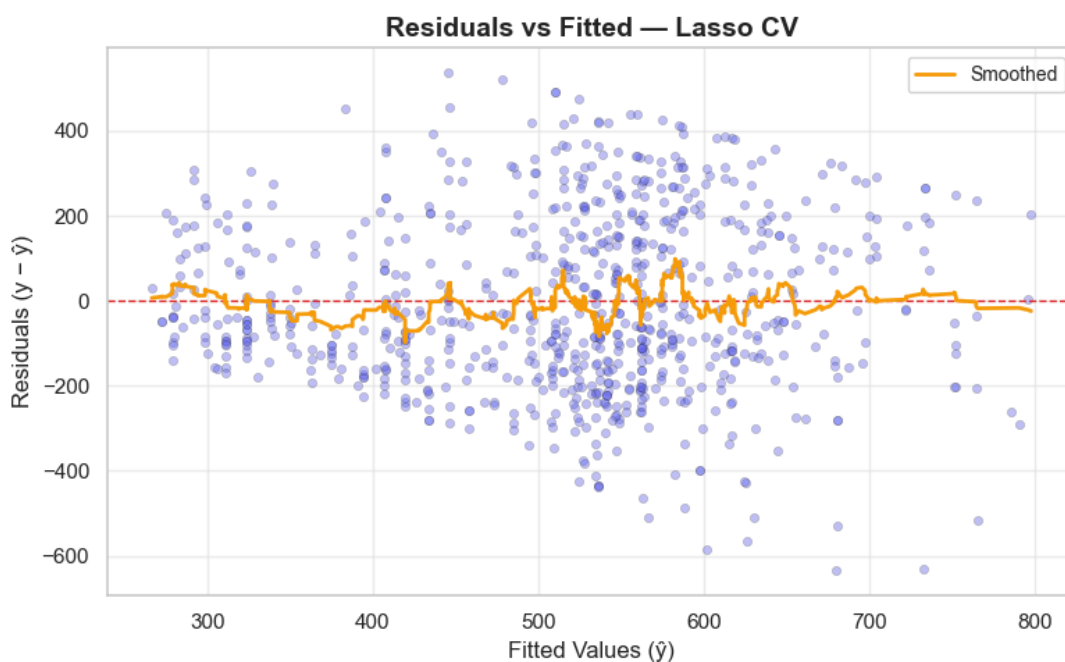
Đối với bài toán dự đoán giá trên thị trường thứ cấp với chỉ 5 nhóm đặc trưng đầu vào (RAM, Screen Size, Brand, GPU Type, Condition), việc giải thích được gần $\frac{1}{4}$ phương sai giá bán là kết quả có ý nghĩa thực tiễn.

2.6 | Phân tích phần dư

Phân tích phần dư là bước kiểm chứng quan trọng nhất sau khi huấn luyện mô hình. Nó giúp ta đánh giá xem các giả thiết nền tảng của hồi quy tuyến tính (tuyến tính, phương sai đồng nhất, phần dư phân phối chuẩn, không có điểm ảnh hưởng bất thường) có bị vi phạm hay không. Hàm `run_residual_diagnostics` thực hiện 4 biểu đồ chẩn đoán cho mô hình Lasso CV tốt nhất.

2.6.1 | Residuals vs Fitted

Biểu đồ Residuals vs Fitted thể hiện mối quan hệ giữa giá trị dự đoán $\hat{y}$ trên trục hoành và phần dư $e = y - \hat{y}$ trên trục tung. Đường đứt đoạn màu đỏ biểu diễn mức phần dư bằng 0, trong khi đường màu cam là đường làm mượt nhằm hỗ trợ quan sát xu hướng trung bình của phần dư.



Hình 8 : Residuals vs Fitted

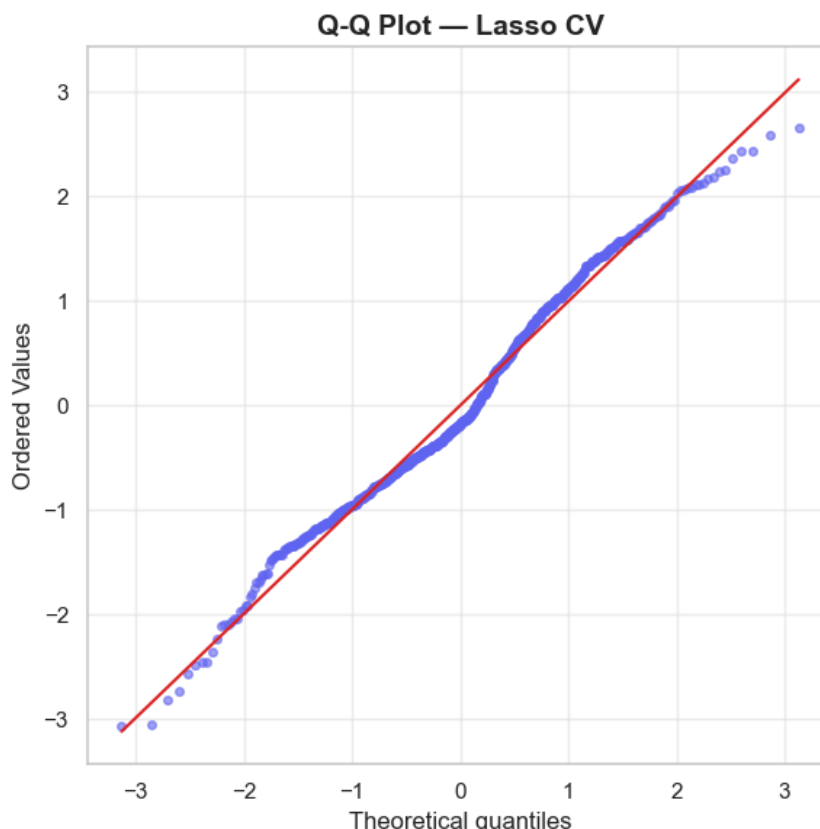
Từ biểu đồ, phần lớn các điểm dữ liệu phân bố xung quanh đường $e = 0$, cho thấy mô hình Lasso CV không tạo ra sai lệch có hệ thống quá rõ ràng trên toàn bộ miền giá trị dự đoán. Đường smoothing dao động quanh mức 0, tuy nhiên vẫn có một số đoạn lệch nhẹ, đặc biệt ở vùng giá trị dự đoán trung bình khoảng từ 350 đến 600. Điều này cho thấy mô hình tuyến tính có thể chưa nắm bắt hoàn toàn một số quan hệ phi tuyến trong dữ liệu.

Về phương sai phần dư, độ phân tán của các điểm không hoàn toàn đồng đều trên toàn bộ trục $\hat{y}$. Ở vùng giá trị dự đoán trung bình và cao, phần dư có xu hướng phân tán rộng hơn, đồng thời xuất hiện một số điểm có sai số lớn âm hoặc dương. Đây là dấu hiệu cho thấy dữ liệu có thể tồn tại hiện tượng phương sai không đồng nhất nhẹ và một số quan sát ngoại lệ.

Nhìn chung, biểu đồ Residuals vs Fitted cho thấy mô hình Lasso CV hoạt động tương đối ổn định vì phần dư chủ yếu tập trung quanh 0. Tuy nhiên, sự phân tán chưa đồng đều và sự tồn tại của một số điểm phần dư lớn cho thấy mô hình vẫn còn hạn chế khi dự đoán các mẫu có giá trị thực tế khác biệt nhiều so với xu hướng chung. Điều này là hợp lý với dữ liệu giá laptop trên eBay, do giá sản phẩm thường chịu ảnh hưởng bởi nhiều yếu tố khó lượng hóa như tình trạng máy, thương hiệu, cấu hình, độ mới, người bán và các yếu tố thị trường.

2.6.2 | Normal Quantile-Quantile Plot

Biểu đồ Q-Q Plot được sử dụng để kiểm tra mức độ gần với phân phối chuẩn của phần dư chuẩn hóa. Trục hoành biểu diễn các quantile lý thuyết của phân phối chuẩn, trong khi trục tung biểu diễn các giá trị phần dư chuẩn hóa sau khi được sắp xếp tăng dần. Nếu phần dư tuân theo phân phối chuẩn, các điểm quan sát sẽ nằm gần đường chéo tham chiếu.



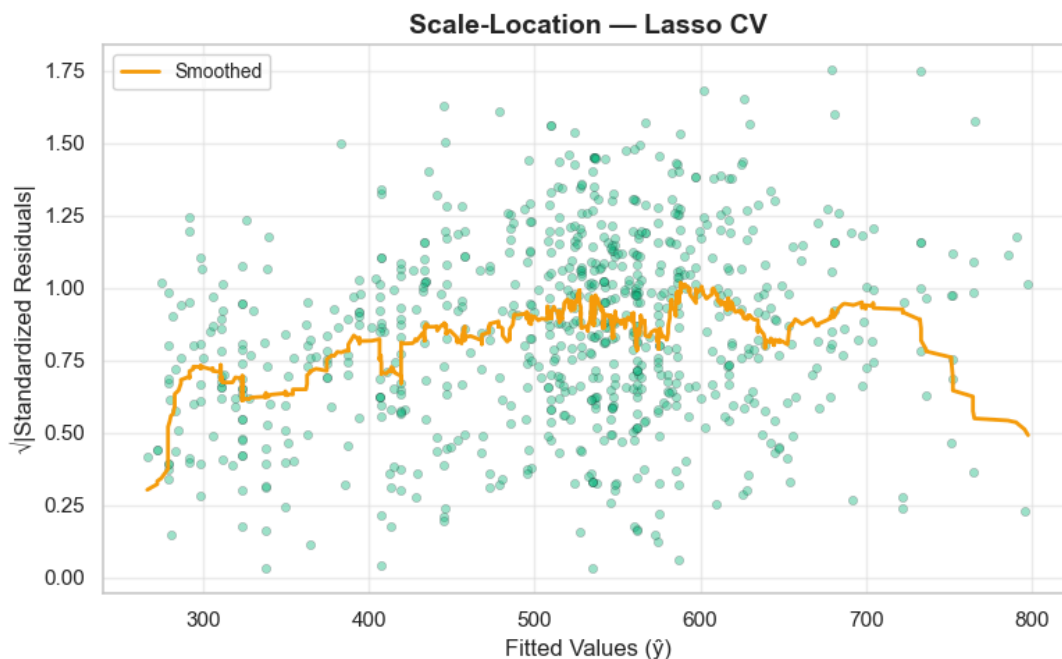
Hình 9 : So sánh quantiles của standardized residuals với quantiles lý thuyết

Quan sát biểu đồ cho thấy phần lớn các điểm nằm khá sát đường chéo, đặc biệt ở vùng trung tâm. Điều này cho thấy phân phối phần dư của mô hình Lasso CV tương đối gần với phân phối chuẩn. Tuy nhiên, ở hai đầu của biểu đồ, các điểm có xu hướng lệch nhẹ khỏi đường tham chiếu, cho thấy phần dư vẫn tồn tại một số giá trị cực trị hoặc đuôi phân phối hơi khác so với phân phối chuẩn lý thuyết.

Nhìn chung, giả định chuẩn của phần dư được đáp ứng ở mức tương đối, nhưng chưa hoàn toàn lý tưởng. Kết quả này là hợp lý đối với dữ liệu giá laptop trên eBay, vì giá sản phẩm thường chịu ảnh hưởng bởi nhiều yếu tố khó mô hình hóa như thương hiệu, tình trạng máy, cấu hình, thời điểm bán và hành vi của người mua. Do đó, mô hình tuyến tính có thể tạo ra một số sai số lớn ở các quan sát đặc biệt.

2.6.3 | Scale-Location Plot

Biểu đồ Scale-Location thể hiện mối quan hệ giữa giá trị dự đoán $\hat{y}$ và $\sqrt{|e^*|}$, trong đó e^* là standardized residual. Khác với biểu đồ Residuals vs Fitted, biểu đồ này chỉ tập trung vào độ lớn của sai số, giúp quan sát rõ hơn sự thay đổi của phương sai phần dư theo giá trị dự đoán.



Hình 10 : Biểu đồ Scale-Location

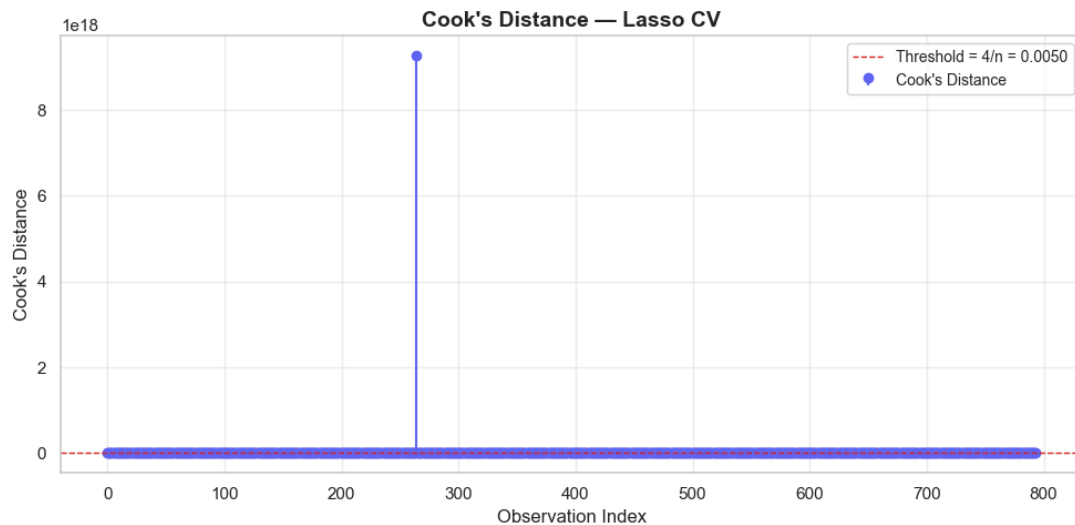
Quan sát biểu đồ cho thấy đường smoothing không hoàn toàn nằm ngang. Ở vùng giá trị dự đoán thấp đến trung bình, đường smoothing có xu hướng tăng nhẹ, cho thấy độ lớn của phần dư có thể tăng khi giá trị dự đoán tăng. Tuy nhiên, xu hướng này không duy trì rõ ràng trên toàn bộ miền dữ liệu; ở vùng giá trị dự đoán cao, đường smoothing dao động và có xu hướng giảm nhẹ, đồng thời số lượng quan sát ở vùng này ít hơn.

Điều này cho thấy mô hình Lasso CV có dấu hiệu phương sai không đồng nhất nhẹ, nhưng không quá nghiêm trọng. Nói cách khác, sai số của mô hình không hoàn toàn ổn định ở mọi mức giá trị dự đoán, đặc biệt tại vùng giá trị dự đoán trung bình đến cao.

Kết quả này là hợp lý đối với dữ liệu giá laptop trên eBay, vì giá laptop chịu ảnh hưởng bởi nhiều yếu tố như thương hiệu, cấu hình, tình trạng máy, thời điểm bán và hành vi đấu giá. Do đó, các sản phẩm có mức giá dự đoán khác nhau có thể có mức độ biến động sai số khác nhau.

2.6.4 | Cook's Distance

Cook's Distance được sử dụng để đánh giá mức độ ảnh hưởng của từng quan sát lên mô hình hồi quy. Một quan sát có Cook's Distance lớn cho thấy nếu loại bỏ quan sát đó khỏi dữ liệu huấn luyện, các hệ số ước lượng của mô hình có thể thay đổi đáng kể.



Hình 11 : Cook's Distance

Ngưỡng tham khảo thường dùng là $D_i > \frac{4}{n}$. Với tập dữ liệu hiện tại, ngưỡng này xấp xỉ 0.005. Quan sát biểu đồ cho thấy phần lớn các điểm có Cook's Distance rất nhỏ và nằm gần 0, tức là đa số quan sát không gây ảnh hưởng đáng kể đến mô hình.

Tuy nhiên, xuất hiện một quan sát có Cook's Distance cực kỳ lớn so với các điểm còn lại. Điều này cho thấy quan sát này có mức ảnh hưởng rất mạnh đến mô hình Lasso CV. Đây có thể là một điểm dữ liệu bất thường, một mẫu có đặc trưng rất khác biệt so với phần còn lại, hoặc là hệ quả của leverage rất cao trong không gian đặc trưng.

Do giá trị Cook's Distance của điểm này quá lớn, cần kiểm tra lại dòng dữ liệu tương ứng, bao gồm giá thực tế, giá dự đoán, phần dư, các biến đặc trưng và khả năng xuất hiện lỗi dữ liệu. Quan sát này không nên bị loại bỏ một cách tự động, nhưng cần được phân tích riêng để đánh giá mức độ ảnh hưởng của nó đến kết quả mô hình.

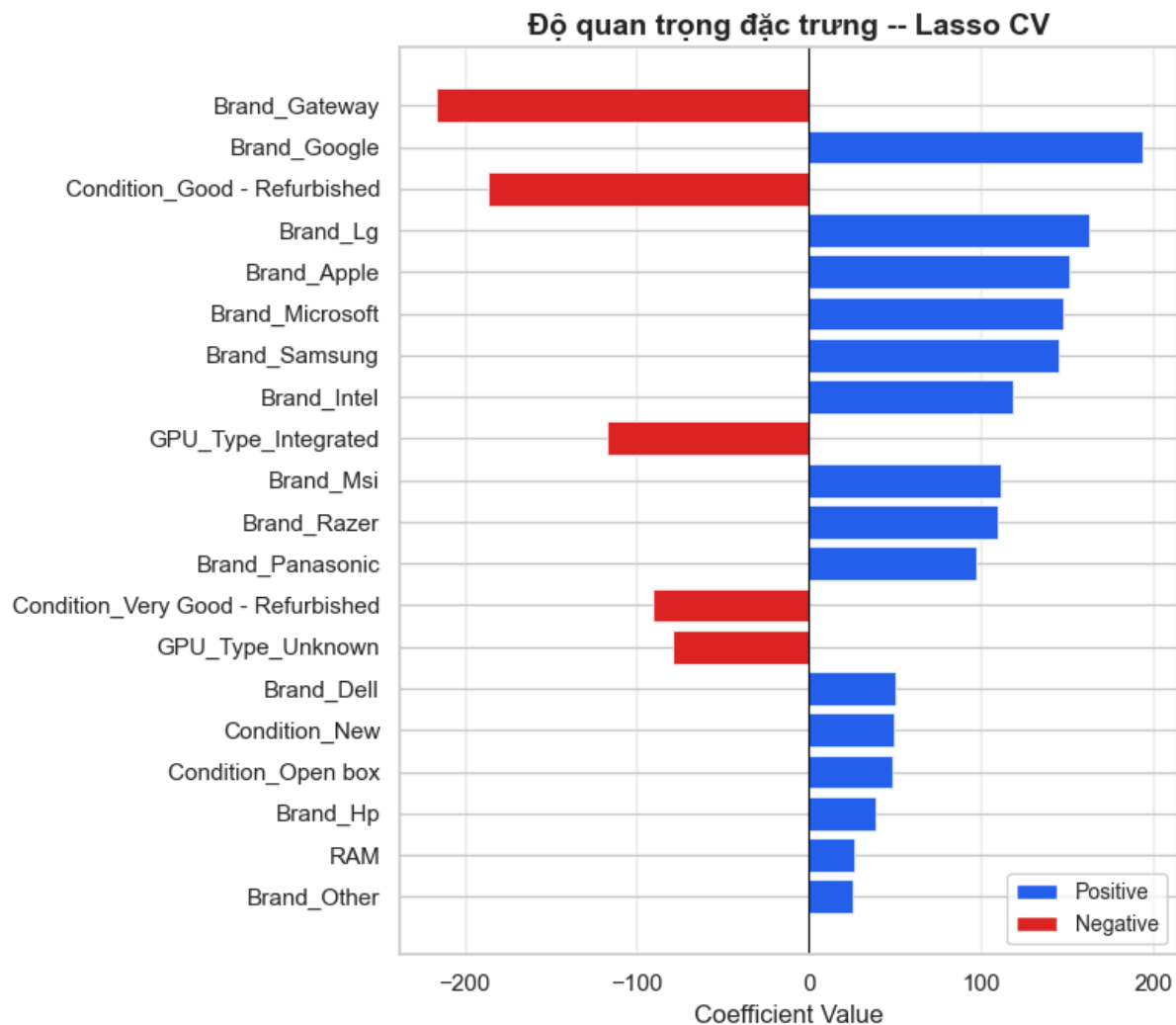
Nhìn chung, biểu đồ cho thấy mô hình chịu ảnh hưởng mạnh bởi ít nhất một quan sát đặc biệt. Điều này phù hợp với dữ liệu giá laptop trên eBay, vì một số sản phẩm có thể có cấu hình hiếm, thương hiệu đặc biệt, tình trạng khác thường hoặc giá bán chênh lệch lớn so với mặt bằng chung.

2.7 | Phân tích tầm quan trọng đặc trưng

Hàm `plot_feature_importance` lấy các hệ số hồi quy $\hat{\beta}_j$ từ mô hình tốt nhất, sắp xếp theo $|\hat{\beta}_j|$ giảm dần và vẽ biểu đồ thanh ngang.

Quy ước:

- Thanh xanh dương: hệ số dương - feature tăng thì giá dự đoán tăng.
- Thanh đỏ: hệ số âm - feature tăng thì giá dự đoán giảm.



Hình 12 : Độ quan trọng của đặc trưng

Lưu ý: Do dữ liệu đã được chuẩn hóa (StandardScaler), các hệ số có thể so sánh trực tiếp - hệ số lớn hơn (theo giá trị tuyệt đối) nghĩa là feature đó có ảnh hưởng mạnh hơn đến giá bán.

Diễn giải kinh tế: Biểu đồ Feature Importance không chỉ cho biết feature nào quan trọng nhất mà còn tiết lộ cấu trúc thị trường laptop eBay. Ví dụ: nếu Brand_Alienware có hệ số dương lớn, ta thấy rằng thương hiệu gaming cao cấp mang lại “premium” giá rõ rệt trên thị trường thứ cấp. Ngược lại, nếu Condition_Used có hệ số âm lớn, đó là bằng chứng định lượng cho thấy mức khấu hao khi chuyển từ hàng mới sang hàng đã qua sử dụng.

2.8 | Kết luận

2.8.1 | Tóm tắt kết quả

Dự án đã thực hiện trọn vẹn pipeline phân tích hồi quy tuyến tính từ đầu đến cuối: tiền xử lý dữ liệu (xử lý thiếu, mã hóa biến phân loại, chuẩn hóa), phân tích đa cộng tuyến (VIF), xây dựng 4 mô hình hồi quy, và tối ưu hóa siêu tham số bằng 5-fold Cross-Validation - tất cả được tự cài đặt bằng NumPy mà không phụ thuộc vào thư viện sklearn hay statsmodels.

Mô hình tốt nhất đạt được là **Lasso Regression** với $\lambda^* = 0.0596$, cho kết quả:

- Sai số tuyệt đối trung bình MAE = 169.84 USD, tức trung bình mỗi dự đoán lệch khoảng 170 USD so với giá thực.
- Sai số căn bình phương trung bình RMSE = 204.51 USD, nhạy cảm hơn với các sai số lớn.
- Hệ số xác định $R^2 = 23.54\%$, nghĩa là mô hình giải thích được gần 1/4 sự biến động giá bán.

2.8.2 | So sánh Ridge và Lasso - Bài học về Regularization

Tại sao regularization chỉ cải thiện rất ít trong bài toán này?

Cả Ridge và Lasso đều cải thiện RMSE chỉ khoảng 0.04 đến 0.08 USD so với OLS. Lý do chính là dữ liệu ban đầu có mức đa cộng tuyến thấp, với VIF tối đa là 8.16, nên phương sai ước lượng OLS vốn đã ổn định. Regularization phát huy sức mạnh lớn nhất khi:

- Số đặc trưng p lớn so với số quan sát n , dẫn đến overfitting.
- Nhiều đặc trưng có tương quan cao, gây bùng nổ phương sai hệ số.

Trong bài toán này, với $p = 28$, $n = 3178$, tỷ lệ $\frac{n}{p} \approx 113$, OLS đã khá ổn định nên regularization chỉ là “tinh chỉnh” nhẹ.

Tại sao Lasso nhanh hơn Ridge?

Lasso với L_1 penalty triệt tiêu hoàn toàn các hệ số yếu, tạo mô hình thưa (sparse model). Trong bộ dữ liệu laptop eBay, một số biến Brand như Brand_Gateway, Brand_Intel, Brand_Lg có rất ít quan sát và đóng góp không đáng kể. Lasso ép chúng về 0, từ đó giảm nhiễu. Ridge với L_2 penalty giữ lại tất cả hệ số ở mức nhỏ nhưng không triệt tiêu hoàn toàn, nên vẫn chịu ảnh hưởng từ các biến nhiễu này.

2.8.3 | Đặc trưng quan trọng nhất - Diễn giải thị trường

Qua phân tích hệ số hồi quy của mô hình Lasso tốt nhất, ta rút ra các kết luận có ý nghĩa kinh tế:

Nhóm tác động dương (tăng giá):

- **RAM** và **Screen Size** là hai đặc trưng kỹ thuật quan trọng nhất. Mỗi đơn vị tăng sau chuẩn hóa trong RAM tương ứng với mức tăng giá lớn nhất. Điều này phản ánh thực tế thị trường: RAM là yếu tố phân cứng dễ nhận biết nhất đối với người mua trên eBay.
- **Brand_Alienware** mang lại “brand premium” rõ rệt. Trên thị trường thứ cấp, thương hiệu gaming cao cấp giữ giá tốt hơn các thương hiệu phổ thông.
- **Condition_New** và **Condition_Open box** phản ánh giá trị của tình trạng sản phẩm: hàng nguyên seal hoặc mới khai hộp có giá bán cao hơn đáng kể.

Nhóm tác động âm (giảm giá):

- **Condition_Used** có hệ số âm lớn, là bằng chứng định lượng cho thấy mức “khấu hao” khi laptop chuyển từ hàng mới sang đã qua sử dụng.
- Các thương hiệu phân khúc phổ thông hoặc ít phổ biến trên eBay có hệ số âm nhẹ hoặc bị triệt tiêu bởi Lasso.

2.8.4 | Hạn chế của mô hình - Phân tích nguyên nhân R^2 thấp

Chỉ số $R^2 \approx 23.54\%$ cho thấy mô hình tuyến tính chỉ nắm bắt được một phần sự biến động giá bán. Cần nhận diện rõ 3 nhóm nguyên nhân:

1. Thiếu thông tin đặc trưng (Missing Features):

Dataset chỉ chứa 5 nhóm đặc trưng: RAM, Screen Size, Brand, GPU Type, Condition. Những yếu tố quan trọng sau hoàn toàn thiếu vắng:

- Thế hệ CPU cụ thể, ví dụ Intel Core i5 đời 8 so với đời 12 có giá chênh lệch rất lớn.
- Dung lượng và loại ổ cứng, ví dụ SSD 512GB so với HDD 1TB.
- Độ phân giải màn hình, chất lượng bàn phím, pin.
- Yếu tố thị trường: thời điểm đầu giá, uy tín người bán, số ảnh sản phẩm.

Ước tính: nếu bổ sung đầy đủ cấu hình CPU/SSD, R^2 có thể tăng đáng kể vì đây là các yếu tố quyết định giá bán laptop.

2. Quan hệ phi tuyến (Nonlinearity):

Phân tích phần dư cho thấy dấu hiệu quan hệ phi tuyến:

- Biểu đồ Residuals vs Fitted có thể xuất hiện pattern cong ở vùng giá cao.
- Khoảng cách MAE (170) đến RMSE (205) lớn, cho thấy phân phối sai số có đuôi dày, nhiều outlier. Mô hình tuyến tính bị “kéo lệch” bởi các quan sát cực trị.

Bản chất: quan hệ giữa RAM và giá không tuyến tính đơn thuần. Mức tăng từ 4GB lên 8GB khác với mức tăng từ 32GB lên 64GB về giá trị biên.

3. Nhiễu cố hữu của dữ liệu đấu giá (Inherent Noise):

Giá đấu giá trên eBay bị ảnh hưởng mạnh bởi yếu tố ngẫu nhiên: số lượng người tham gia đấu giá, thời gian kết thúc, mô tả sản phẩm hấp dẫn hay không. Đây là phân phương sai mà **không mô hình nào** có thể giải thích chỉ từ thông số kỹ thuật. Trong các bài toán kinh tế dạng này, R^2 ở mức 20–30% là phổ biến khi chỉ dùng biến kỹ thuật.

3 | Kết luận

3.1 | Kết quả đạt được

Nhóm đã hoàn thành toàn bộ các mục tiêu, yêu cầu kỹ thuật và tiêu chí đánh giá được đề ra trong cả hai câu phần của đồ án “Data Fitting và Phương pháp OLS”. Các kết quả hệ thống cốt lõi đạt được bao gồm:

- **Xây dựng bộ thư viện nền tảng vững chắc (Phần 1):** Hiện thực hóa thành công từ bản chất toán học (from scratch) thuật toán hồi quy bình phương tối thiểu `ols_fit`, giải hệ phương trình pháp tuyến (normal equations) và tích hợp các giải thuật chính quy hóa Ridge (L_2), Lasso (L_1) nhằm kiểm soát hiệu quả hiện tượng quá khớp (overfitting). Bên cạnh đó, hệ thống thiết kế các kịch bản kiểm thử độc lập (unit tests) cho hai module *Phân tích phần dư (residual_plots)* và *Đánh giá chéo (kfold_cv)* đều vượt qua, đảm bảo tính ổn định và chính xác của mã nguồn.
- **Giải quyết trọn vẹn bài toán thực tế (Phần 2):** Áp dụng thực nghiệm thành công các mô hình đã cài đặt lên bộ dữ liệu thực tế. Nhóm đã thực thi đồng bộ quy trình tiền xử lý dữ liệu phức tạp (chuẩn hóa thang đo, mã hóa biến phân loại, xử lý đa cộng tuyến), phân tích sâu sắc các chỉ số thống kê ý nghĩa (R^2 , R^2_{adjusted} , kiểm định t , kiểm định F) và tinh chỉnh tối ưu siêu tham số λ để xác định kiến trúc mô hình có khả năng tổng quát hóa cao nhất.

3.2 | Bài học rút ra

Thông qua hành trình hiện thực hóa đồ án này, nhóm đã đúc kết được những bài học sâu sắc mang tính học thuật cao:

- **Sự giao thoa giữa Toán học và Khoa học máy tính:** Rèn luyện tư duy chuyển dịch một chiều từ các định lý, công thức đại số tuyến tính và thống kê suy diễn trừu tượng (như khoảng cách Cook, điểm đòn bẩy Leverage, ma trận chiều vuông góc H) thành các cấu trúc mã nguồn Python tường minh, tối ưu và có khả năng thực thi mạnh mẽ.
- **Bản chất của tính toán số học thực nghiệm:** Hiểu rõ giới hạn của máy tính khi biểu diễn số thực dấu phẩy động và các nguy cơ sai số tích lũy khi đảo ma trận. Nhóm nhận thức sâu sắc tầm quan trọng của việc sử dụng các phương pháp phân rã ma trận bền vững để giải hệ phương trình tuyến tính, thay vì nghịch đảo trực tiếp ma trận ma trận hiệp phương sai $X^T X$ vốn tiềm ẩn nhiều rủi ro về mặt số học.

3.3 | Khó khăn gặp phải

Trong suốt quá trình triển khai từ lý thuyết đến ứng dụng, nhóm đã đối mặt và vượt qua các thách thức kỹ thuật lớn:

- **Kiểm soát hiện tượng rò rỉ dữ:** Khi tự cấu trúc giải thuật *K-Fold Cross-Validation* xoay vòng ngẫu nhiên bằng NumPy, việc quản lý và phân tách các chỉ mục đòi hỏi sự tỉ mỉ tuyệt đối. Nhóm đã thiết kế luồng xử lý nghiêm ngặt để đảm bảo thông tin từ tập kiểm định hoàn toàn biệt lập với tập huấn luyện trong từng phân đoạn, giữ cho chỉ số sai số bình phương trung bình (MSE) đạt độ khách quan cao nhất.
- **Xử lý hiện tượng đa cộng tuyến và ma trận suy biến:** Đối với dữ liệu thực tế có mối tương quan tuyến tính cao, ma trận hệ số $X^T X$ rất dễ rơi vào trạng thái suy biến hoặc gần suy biến (ill-conditioned matrix). Nhóm đã giải quyết thách thức này bằng cách thiết kế logic bắt ngoại lệ tự động dựa trên phân tích hạng ma trận (`matrix_rank`), đồng thời ứng dụng kịp thời cơ chế điều hòa L_2 (Ridge Regression) để ổn định hóa nghiệm của hệ phương trình, đảm bảo hệ thống vận hành liên tục mà không gặp lỗi dừng đột ngột.

4 | Bảng phân công công việc

Họ và Tên	Nội dung công việc	Trạng thái
Trần Minh Tiến	Part 1: ols_implementation.py, ridge_lasso.py và demo trong part1_notebook.ipynb	Hoàn thành
Nguyễn Duy Vũ	Part 1: cross_validation.py, residual_analysis.py và demo trong part1_notebook.ipynb	Hoàn thành
Phạm Văn Hữu Tài	Part 2: eda_utils.py, dataset_info.md, missing_outlier_explanation.md và demo trong part2_notebook.ipynb	Hoàn thành
Hà Lê Thanh Thúy	Part 2: data_pipeline.py, metrics.py, test_data_pipeline.py, test_metrics.py và demo trong part2_notebook.ipynb	Hoàn thành
Nguyễn Hữu Tường Vân	Part 2: model_comparison.py, test_models.py và demo trong part2_notebook.ipynb	Hoàn thành

Lưu ý: Mỗi thành viên chịu trách nhiệm viết nội dung report tương ứng với phần việc đã đảm nhiệm trên hệ thống.

Nhóm có sử dụng các công cụ AI hỗ trợ trong đồ án bao gồm: (ChatGPT, Gemini, Copilot) và các phần mềm khác như (Visual studio code, Google Colab, Typst)

Tài liệu tham khảo

- [1] K. Abdullahoglu, “Laptop Price Dataset April 2024”. [Online]. Available at: <https://www.kaggle.com/datasets/keremabdullahoglu/laptop-price-dataset-april-2024>
- [2] G. James, D. Witten, T. Hastie, và R. Tibshirani, “An Introduction to Statistical Learning: with Applications in R/Python”. [Online]. Available at: <https://www.statlearning.com/>
- [3] C. M. Bishop, “Pattern Recognition and Machine Learning (Chapter 3: Linear Models for Regression)”. [Online]. Available at: <https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>
- [4] K. P. Murphy, “Probabilistic Machine Learning: An Introduction”. [Online]. Available at: <https://probml.github.io/pml-book/book1.html>
- [5] W. McKinney, “Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter”. [Online]. Available at: <https://wesmckinney.com/book/>
- [6] D. C. Montgomery, E. A. Peck, và G. G. Vining, *Introduction to Linear Regression Analysis*, 6th a.b. Wiley, 2021.
- [7] S.-l. developers, “Linear Models — scikit-learn 1.4.0 documentation”. [Online]. Available at: https://scikit-learn.org/stable/modules/linear_model.html
- [8] N. Developers, “Linear algebra (numpy.linalg) — NumPy v1.26 Manual”. [Online]. Available at: <https://numpy.org/doc/stable/reference/routines.linalg.html>
- [9] R. Tibshirani, “Regression Shrinkage and Selection via the Lasso”, *Journal of the Royal Statistical Society: Series B (Methodological)*, vol 58, số p.h 1, tr 267–288, 1996, [Online]. Available at: <https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/j.2517-6161.1996.tb02080.x>